Lecture 9. Discovering Hidden Structures

Unsupervised Learning: Clustering, Dimensionality Reduction, and Anomaly Detection

DISCOVERING HIDDEN STRUCTURES



Learning Outcomes

What students should be able to do after this lecture

CLUSTERING

Explain K-Means and DBSCAN, choose K, and understand scaling, inertia, silhouette, eps, and noise.

DIMENSIONALITY

Explain the curse of dimensionality, PCA, explained variance, and the purposes of t-SNE and UMAP.

ANOMALIES

Understand Isolation Forest, contamination, and the difference between an anomaly and an actual failure.

Connection to Lecture 8

From predicting a target to exploring the structure of X

WEEK 8 • SUPERVISED

Given: X + y

Question: “How can we predict y?”

Example: customer features → churn / no churn.

Performance can be checked against known labels.

WEEK 9 • UNSUPERVISED

Given: X only

Question: “What structure is hidden in X?”

We look for groups, compact representations, and unusual observations.

What changes without a target?

No predefined correct answers: the problem itself changes



KEY IDEA

Without a target, we cannot simply calculate accuracy. Unsupervised learning requires internal metrics, stability checks, and domain interpretation.
Unsupervised learning is primarily about discovering hidden structure.

Three Families of Tasks

Clustering • Dimensionality Reduction • Anomaly Detection

CLUSTERING

Which observations are similar?

K-Means
DBSCAN

Output: cluster labels.

DIMENSIONALITY

Can the data be described with fewer coordinates?

PCA
t-SNE
UMAP

ANOMALY DETECTION

Which observations are unusual?

Isolation Forest

Output: anomaly score / label.

Lecture Roadmap

2 hours: 40 + 50 + 30 minutes



In each block: clear intuition → visual mechanics → code → mistakes and limitations → real example.

Lecture 9 Terminology

Key terms used throughout this lecture

Centroid

cluster center

Inertia

K-Means compactness

Silhouette

cluster separation

Density

density

Noise

point outside a dense cluster

Dimension

number of features

Principal Component

new PCA axis

Explained Variance

retained variability

Anomaly

unusual observation

What Is Clustering?

Discover natural groups of similar observations

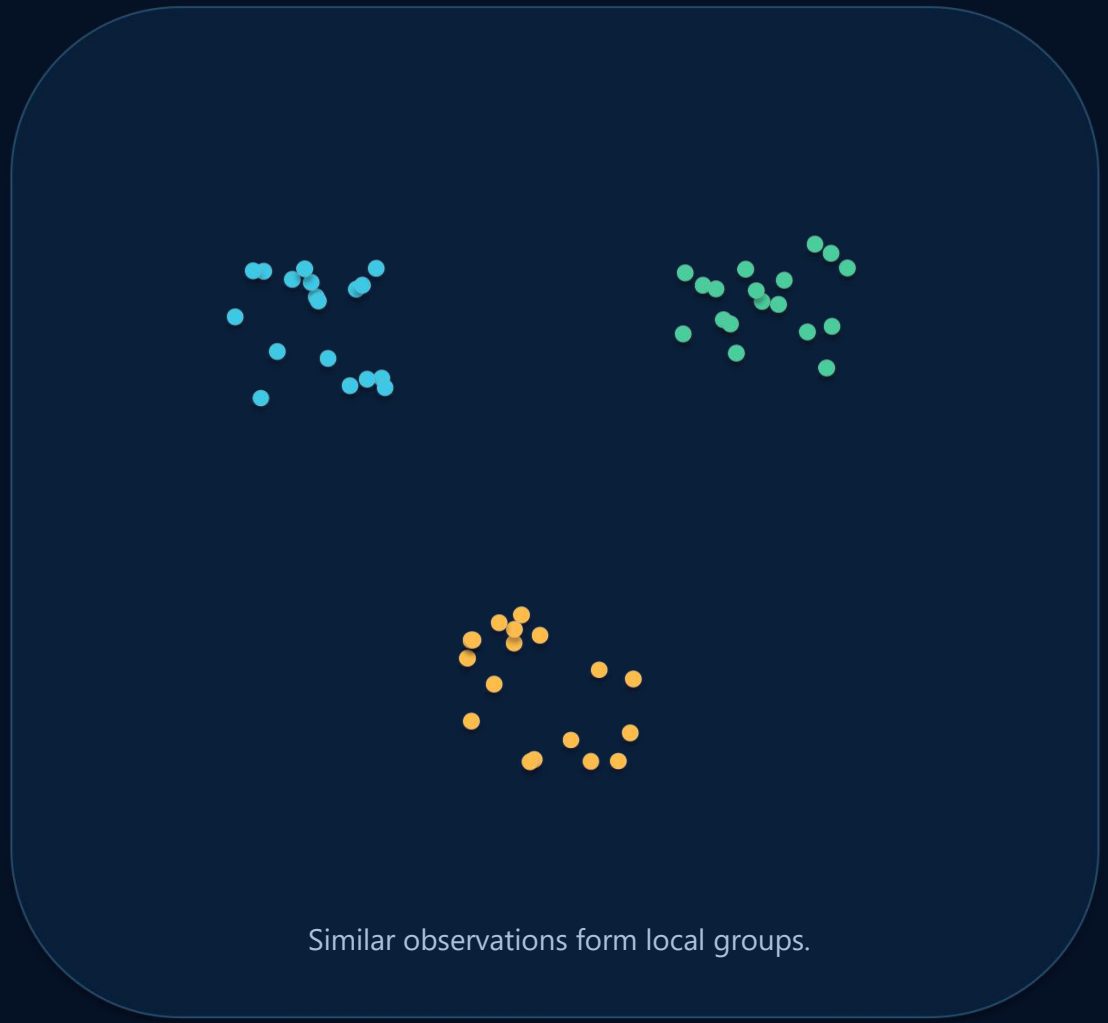
- A cluster is a group of similar observations.

Goal: divide observations into groups so members of each group are similar and different groups are distinct.

- The algorithm does not know the names of the groups.

- Labels 0, 1, 2 are identifiers only.

- People interpret what each cluster means.



Similar observations form local groups.

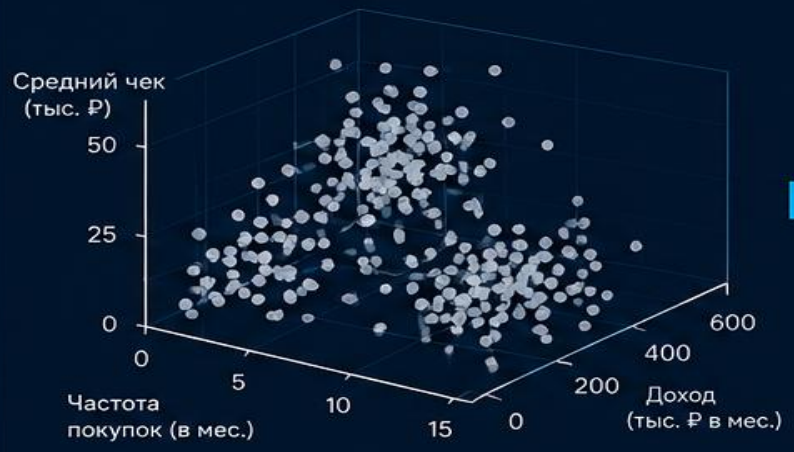
Реальный пример: сегментация клиентов

У нас нет готовых групп клиентов, Алгоритм находит кластеры, а мы исследуем их профиль и даем названия.

Названия кластеров (активные, премиальные, малоактивные) появляются только после анализа характеристик каждого кластера.

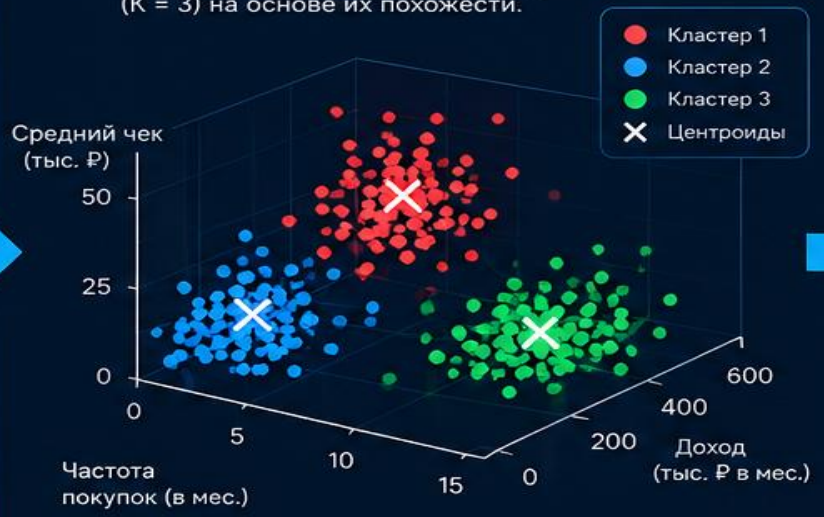
1 Исходные данные

Каждая точка — клиент.
Используем 3 признака:
доход, частота покупок, средний чек.



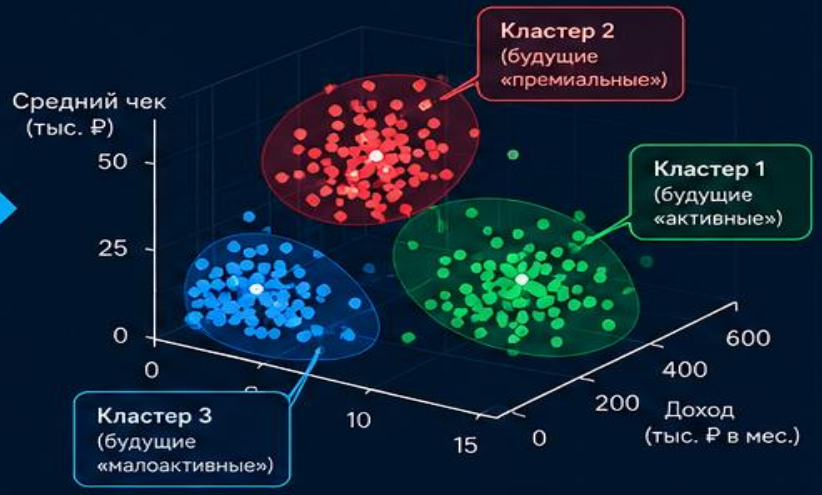
2 K-Means кластеризация

Алгоритм делит клиентов на 3 группы
(K = 3) на основе их похожести.



3 Результат кластеризации

Клиенты разделены на 3 кластера.
Теперь анализируем, чем они отличаются.



4 Анализ профиля кластеров

Смотрим средние значения признаков в каждом кластере.

Показатель	Кластер 1	Кластер 2	Кластер 3
Средний доход (тыс. ₽)	320	820	120
Частота покупок (в мес.)	12	3	2
Средний чек (тыс. ₽)	5	38	4
Размер кластера (% клиентов)	45%	15%	40%

5 Интерпретация и назначение названий

На основе профиля кластера даем понятные названия.
Названия появляются только после анализа данных.

Кластер 1
Активные клиенты

- Средний доход: 320 тыс. ₽
- Частота покупок: 12 в мес.
- Средний чек: 5 тыс. ₽
- Описание: часто покупают, стабильный средний чек.



«Регулярно покупают и вовлечены в наш магазин»

Кластер 2
Премиальные клиенты

- Средний доход: 820 тыс. ₽
- Частота покупок: 3 в мес.
- Средний чек: 38 тыс. ₽
- Описание: высокодоходные клиенты, покупают реже, но с большим средним чеком.



«Меньше покупок, но приносят наибольшую выручку»

Кластер 3
Малоактивные клиенты

- Средний доход: 120 тыс. ₽
- Частота покупок: 2 в мес.
- Средний чек: 4 тыс. ₽
- Описание: покупают редко и с небольшим средним чеком.



«Редкие покупки, небольшой средний чек»

K-Means: Intuition

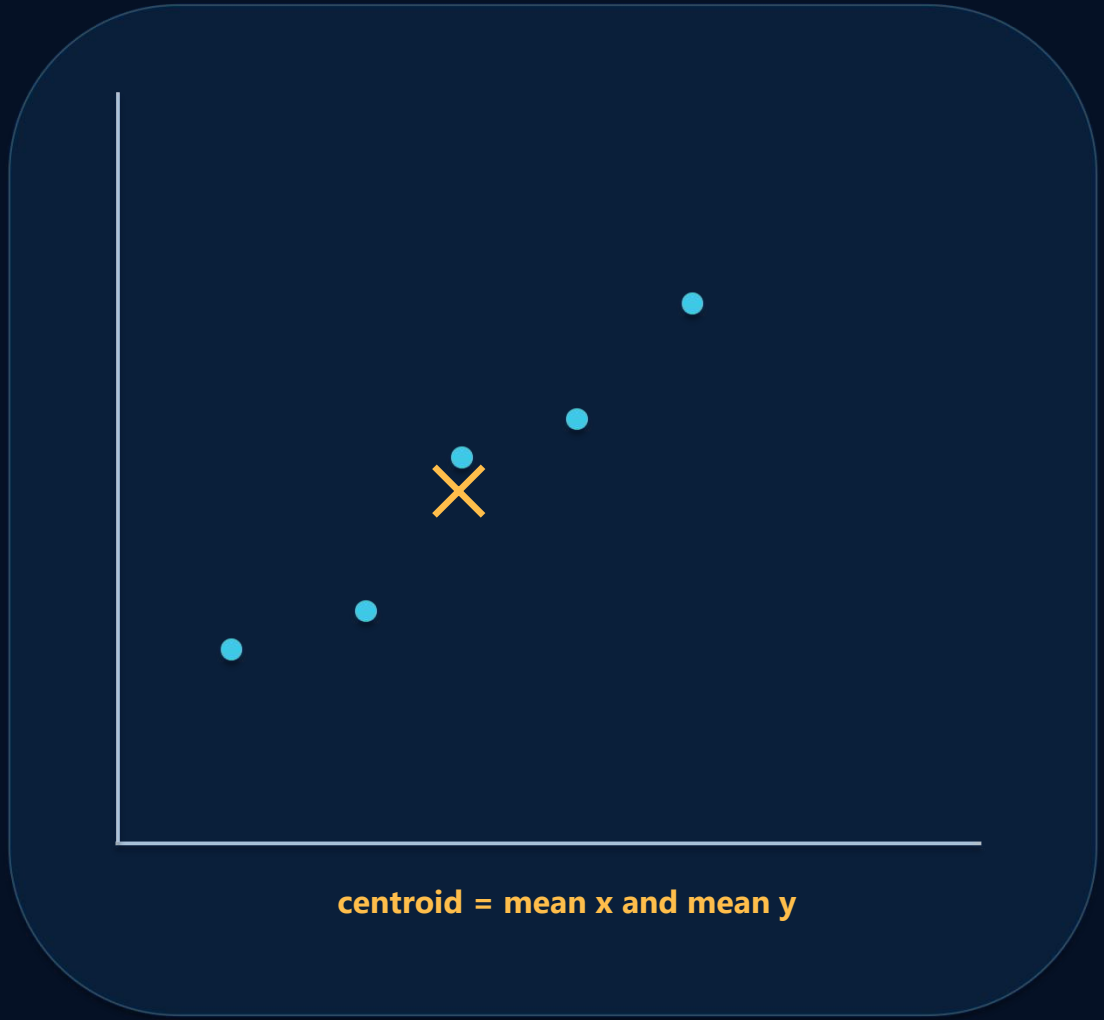
K centers attract their nearest observations

- Choose K: the desired number of groups.
- Initialize K centroids.
- Assign each observation to its nearest center.
- The centers gradually move.

What Is a Centroid?

The mean position of observations within a group

- A centroid is the mean of the coordinates in a group.
- For points (1,2), (2,2), (3,4): C=(2, 2.67).
- A centroid does not have to be an actual observation.



K-Means: Step 1 — Initialization

Choose K and the starting centers

- Choose K, for example K=3.
- Starting centers are initialized automatically.
- Good initialization reduces the risk of a poor local solution.

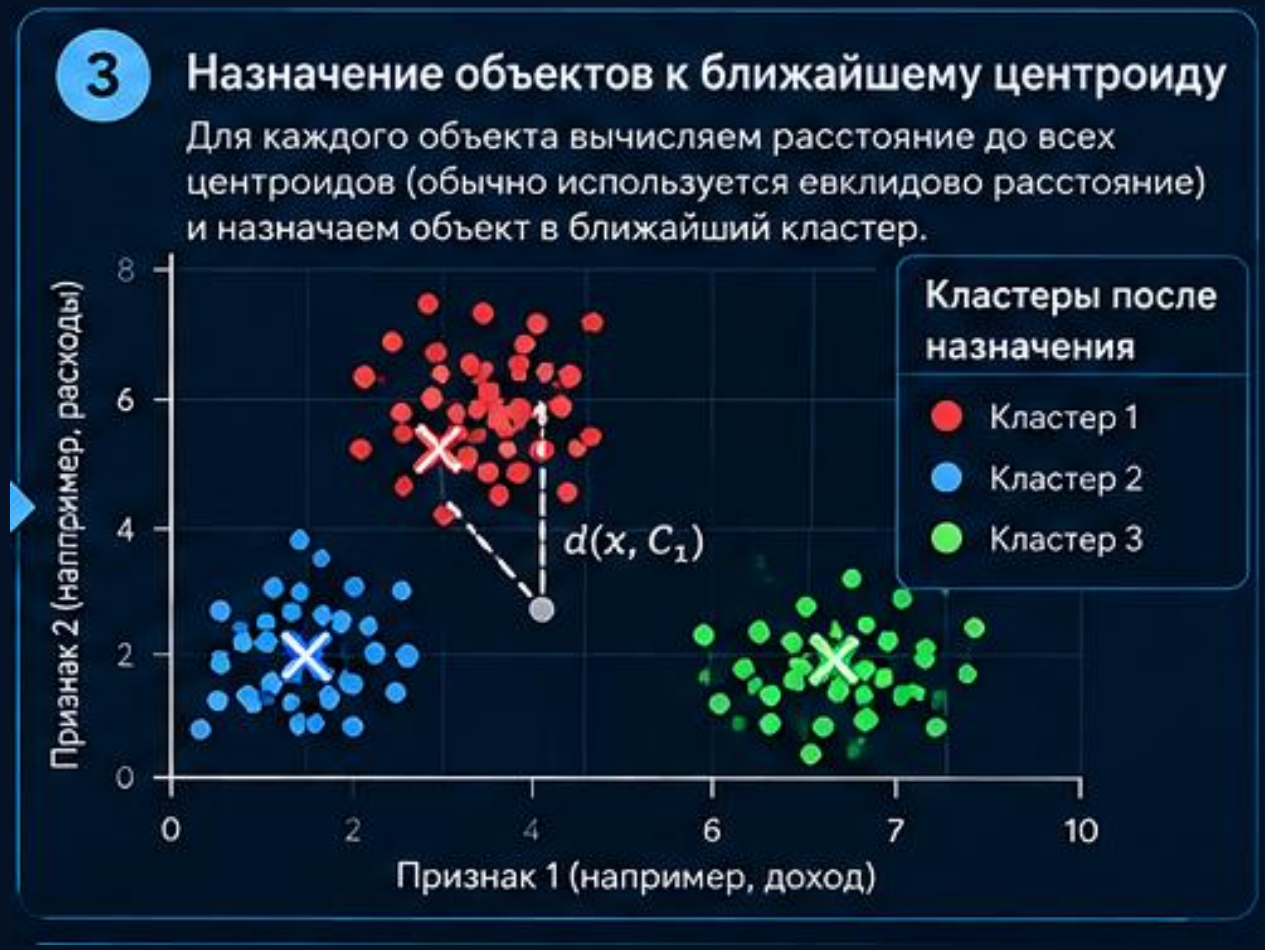




K-Means: Step 2 — Assignment

Each point goes to its nearest centroid

- Calculate distances from each point to the centers.
- Assign the point to the nearest centroid.
- This produces temporary groups.



K-Means: Step 3 — Update

Move each centroid to the mean of its group

- Calculate the mean for each temporary cluster.
- Replace the old centroid with the new mean.
- The center moves to a better position.



K-Means: Step 4 — Convergence

Repeat assignment and update until stable

- Repeat assignment and update.
- Stop when the centers barely change.
- The iterative process converges to a local solution.

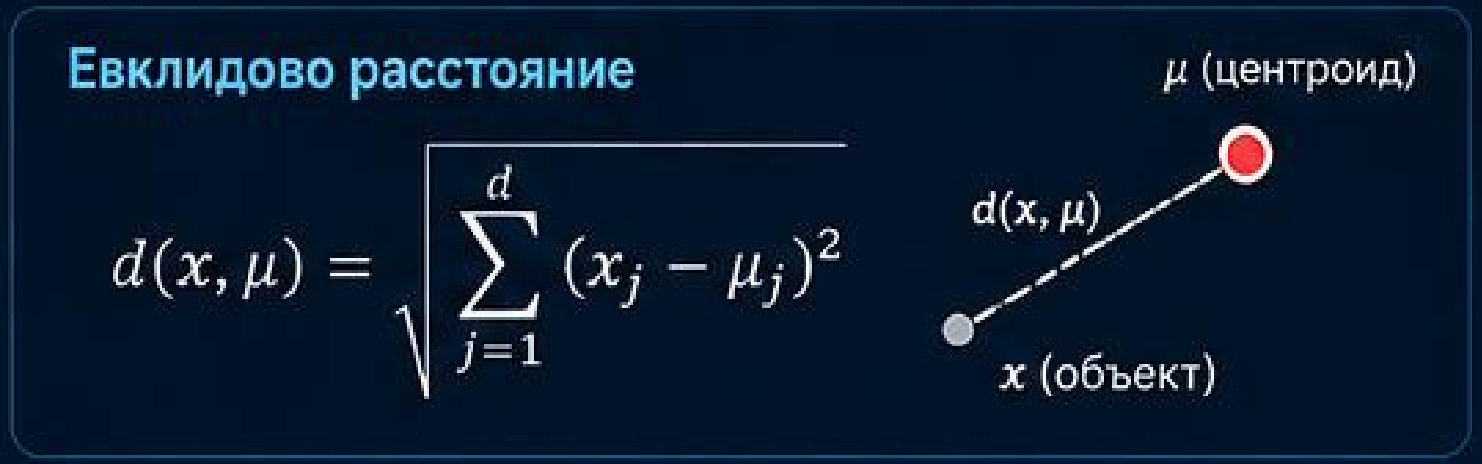




Euclidean Distance

How K-Means determines the nearest observation

- $d(A,B) = \sqrt{\Sigma(Aj - Bj)^2}$
- Small distance → observations are similar under the chosen metric.
- Distance-based methods are sensitive to feature scales.



Why Scaling Is Essential

Age and income should not compete through their units

- Age: 18–80; Income: 100 000–1 000 000.
- Without scaling, income dominates.
- StandardScaler: mean≈0, std≈1.
- After scaling, features have comparable scales.



K-Means in Python

A complete minimal example with StandardScaler

PYTHON

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X_scaled = StandardScaler().fit_transform(X)

model = KMeans(
    n_clusters=3,
    random_state=42,
    n_init="auto"
)
labels = model.fit_predict(X_scaled)
```

WHAT HAPPENS

1. StandardScaler brings features to comparable scales.

2. n_clusters specifies K.

3. fit_predict trains the model and returns a cluster label for each row.

4. After fitting, analyze cluster centers, sizes, and profiles.

What Does K-Means Return?

labels_, cluster_centers_, inertia_

- labels_ → cluster ID for each row.
- cluster_centers_ → centroid coordinates.
- inertia_ → total within-cluster squared error.

Что оптимизирует K-Means?

Алгоритм минимизирует сумму квадратов расстояний объектов до их центроидов (inertia):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- x_i – объект
- μ_k – центроид k-го кластера
- C_k – множество объектов в k-м кластере

Критерий остановки

- Центроиды почти не изменяются
- Максимальное число итераций достигнуто
- Изменение значения функции потерь (inertia) меньше заданного порога

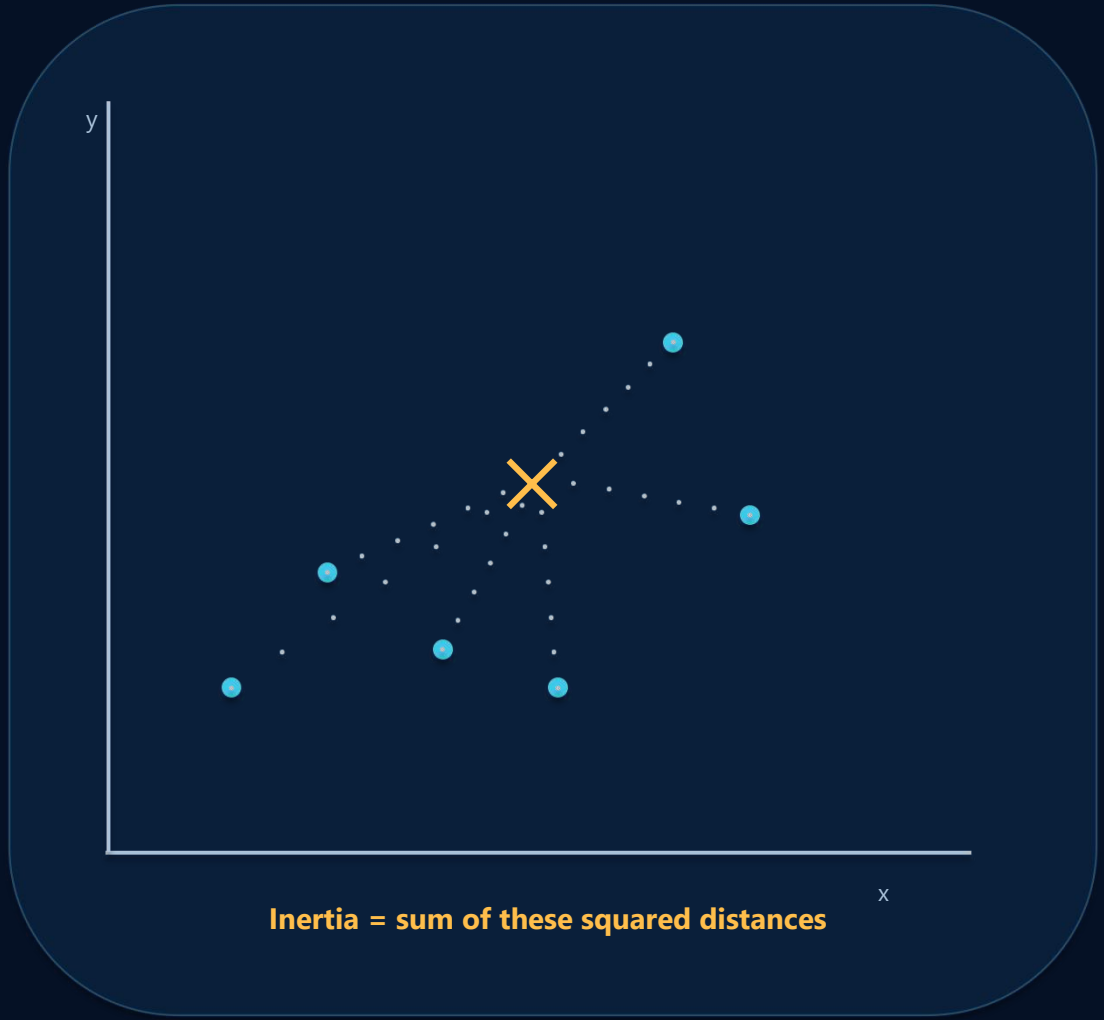
labels_	**[0, 2, 1, 0, ...]**
centers_	**K × p coordinates**
inertia_	**Σ distance²**

Three outputs → three different kinds of interpretation

What Does K-Means Minimize?

Within-cluster sum of squares / inertia

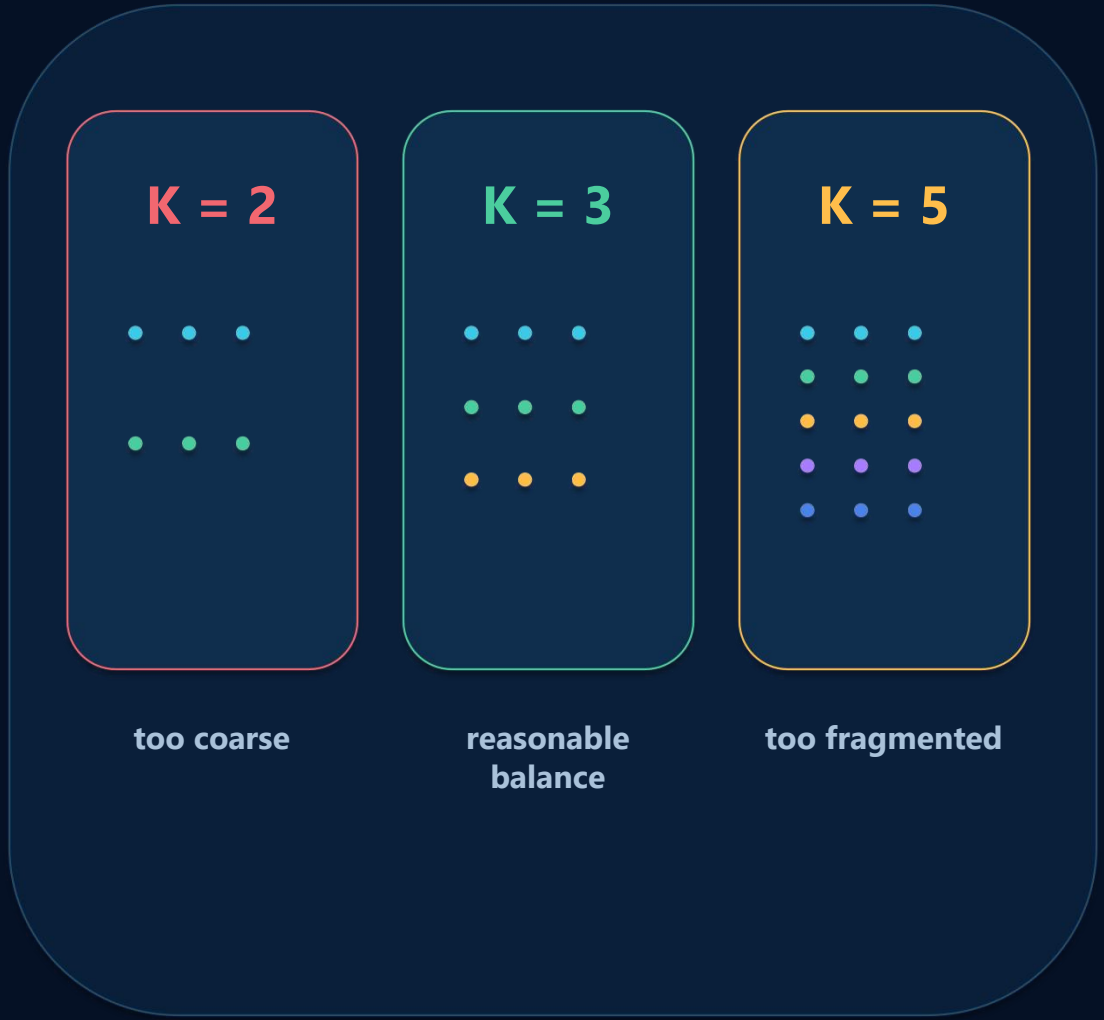
- K-Means minimizes the sum of squared distances to centroids.
- Lower inertia → more compact groups.
- But inertia almost always decreases as K increases.



How Do We Choose K?

K is unknown in advance: use model-based and domain analysis

- There is no universally correct K.
- Try several K values.
- Examine elbow and silhouette.
- Then assess practical meaning.



Как выбрать K? Метод локтя (Elbow Method)

Мы пробуем разные значения K, обучаем K-Means и анализируем, как меняется значение функции функции потерь (inertia).

Идея: добавление нового кластера должно давать существенное улучшение. После некоторого момента улучшение становится незначительным.

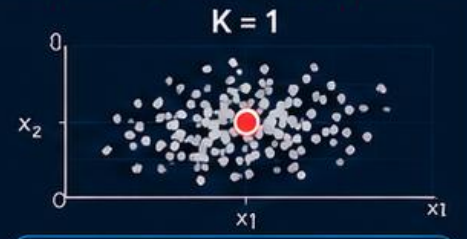
1 Пробуем разные K
Например, K = 1, 2, 3, …, 10.

2 Обучаем K-Means
Для каждого K вычисляем центроиды и значение inertia.

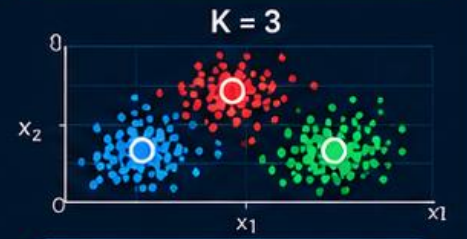
3 Строим график
По оси X — количество кластеров K, по оси Y — inertia.

4 Выбираем «локоть»
Точку, после которой улучшение становится незначительным.

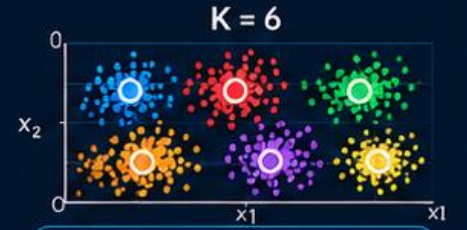
Визуализация кластеров для разных K



Один кластер — большая ошибка (высокая inertia)



Три кластера — значительное улучшение



Шесть кластеров — улучшение уже небольшое

График зависимости inertia от K



Числовые значения inertia для разных K

K	1	2	3	4	5	6	7	8	9	10
Inertia	1280	720	310	250	210	190	175	165	158	150
Улучшение (уменьшение)	—	-560	-410	-60	-40	-20	-15	-10	-7	-8

Формула inertia (функция потерь K-Means)

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- x_i — объект (точка данных)
- μ_k — центроид k-го кластера
- C_k — множество объектов в k-м кластере
- $\|\cdot\|$ — евклидово расстояние

- Inertia — это сумма квадратов расстояний всех объектов до центроидов их кластеров.
- Чем меньше inertia, тем более компактные кластеры.

Интерпретация

- От K = 1 до K = 3 наблюдается большое снижение inertia (значительное улучшение).
- После K = 3 кривая становится «плоской».
- Добавление новых кластеров практически не уменьшает ошибку.

Вывод

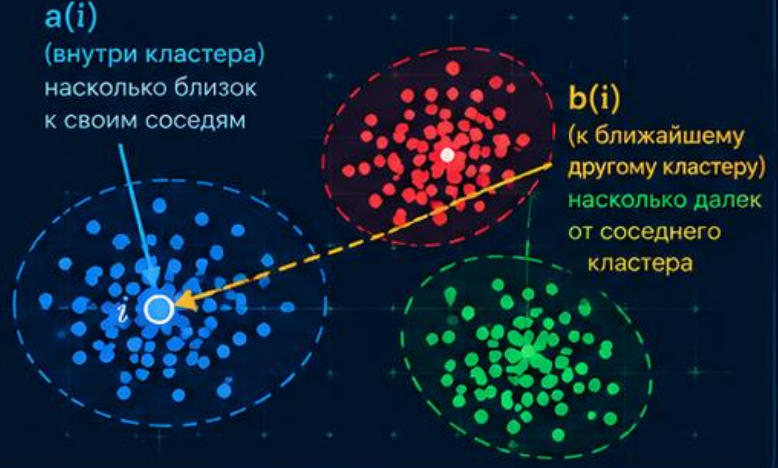
В этом примере оптимальное количество кластеров: K = 3.

Как выбрать K? Метод Silhouette

Silhouette показывает, насколько каждый объект ближе к своему кластеру, чем к соседним кластерам. Значение изменяется от –1 до 1: чем выше, тем лучше разделение кластеров.

Идея: хороший K должен давать кластеры, в которых объекты хорошо похожи внутри кластера и четко отделены от других кластеров.

1 Интуитивная идея



2 Формула Silhouette

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $s(i)$ — значение Silhouette для объекта i
- $a(i)$ — среднее расстояние от i до всех других объектов в своем кластере (внутрикластерная схожесть)
- $b(i)$ — минимальное среднее расстояние от i до объектов ближайшего другого кластера (межкластерное отличие)

3 Интерпретация значений



$s(i) \approx 1$ — Объект хорошо подходит своему кластеру и далеко от других кластеров.



$s(i) \approx 0$ — Объект находится на границе между кластерами.

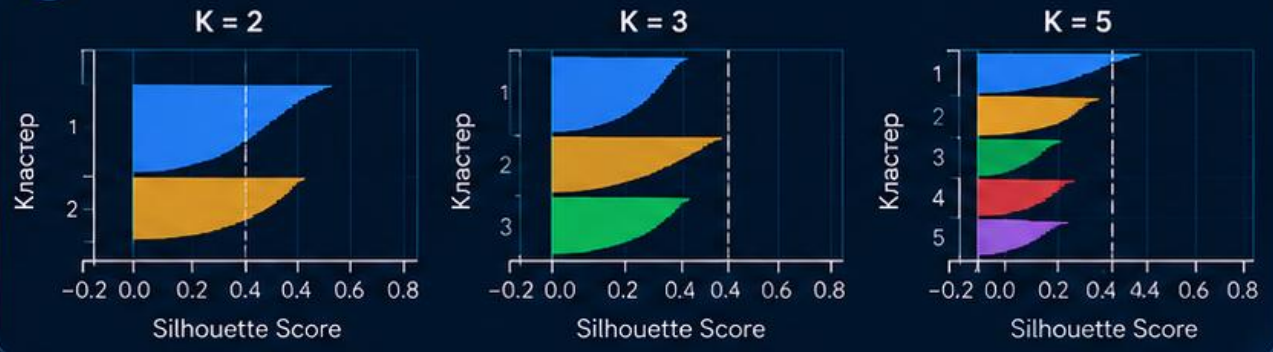


$s(i) < 0$ — Объект ближе к другому кластеру, чем к своему. Возможно, неправильное назначение кластера.

4 График Silhouette для разных K



5 Визуализация для разных K



Вывод

Выбираем то значение K, при котором средний Silhouette Score максимален.

В этом примере оптимальное количество кластеров: **K = 3.**

Audience Question: Is K=10 Better Than K=3?

Why more clusters do not always mean a better model

AUDIENCE QUESTION

If inertia is lower for K=10 than for K=3, does that automatically make K=10 better?

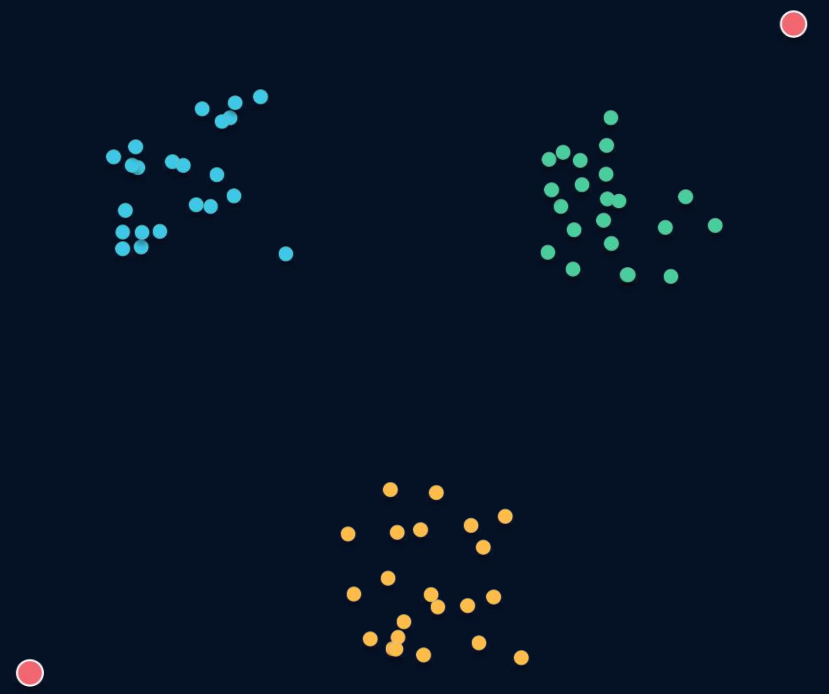
ANSWER / DISCUSSION

No. Inertia almost always falls as K grows. With too many clusters, natural groups may be split into meaningless microclusters. Check elbow, silhouette, stability, and domain meaning.

Limitations of K-Means

Shape, outliers, density, scale, and initialization

- K-Means favors compact clusters.
- Outliers shift the mean.
- Clusters with different densities can be difficult.
- Non-convex shapes may be divided incorrectly.



K-Means limitation: its decision regions are convex polygons. Crescent- or ring-shaped data may therefore be split incorrectly.

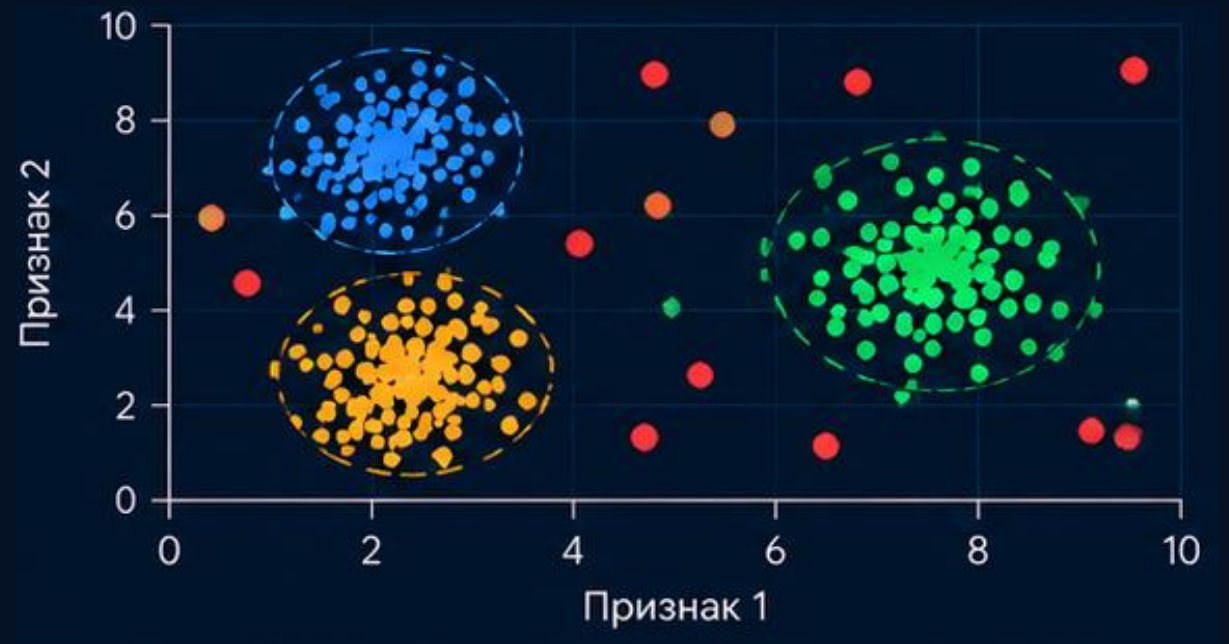
DBSCAN: The Density Idea

Clusters are defined by neighborhoods, not centers

DBSCAN idea: a cluster is a high-density region separated from other regions by low-density areas.

Parameters: eps (neighborhood radius) and min_samples (minimum points within the radius).

Point types: Core, Border, and Noise (label -1).





eps and min_samples

Two parameters defining local density

- eps — radius of the local neighborhood.
- min_samples — minimum number of points for a dense region.
- Small eps → more noise.
- Large eps → groups may merge.

Core, Border, Noise

Three Point Types in DBSCAN

- Core point: has enough neighbors.
- Border point: lies at the edge of a dense region.
- Noise point: is not connected to a dense region.
- In scikit-learn, noise receives label -1.

DBSCAN Step by Step

How a dense region becomes a cluster

- 1. Select an unvisited point.
- 2. Count neighbors within eps.
- 3. If it is a core point, expand the cluster.
- 4. Connect neighboring core points.
- 5. Remaining points may become noise.

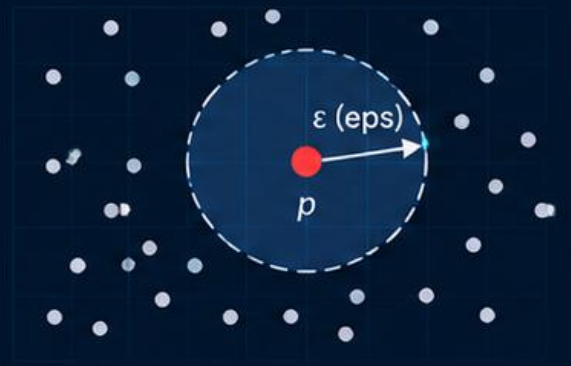
Идея DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN группирует точки, которые находятся в областях высокой плотности, и выделяет точки шума (выбросы), не принадлежащие ни к одному кластеру.

Основная идея:
кластер — это область плотности, отделенная от других областей разреженными зонами.

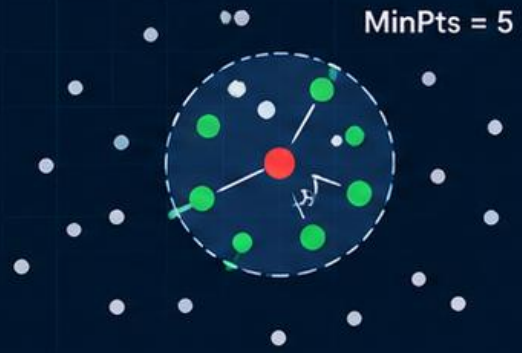
1 Параметры

Для каждой точки рассматриваем окрестность радиуса ε (eps).



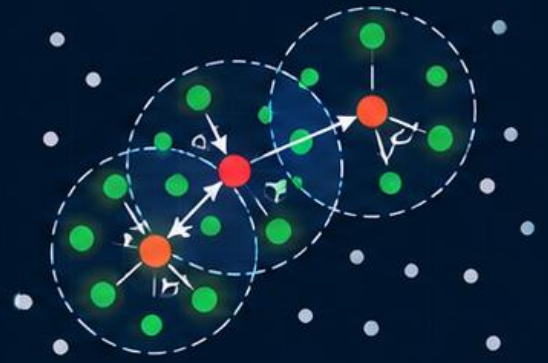
2 Плотность точки

Считаем, сколько соседей находится внутри радиуса ε. Если число соседей ≥ MinPts, точка считается core (ядро).

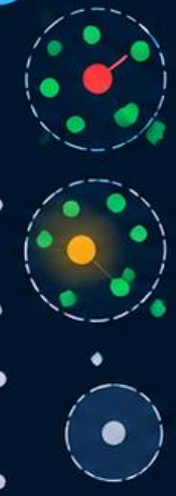


3 Расширение кластера

Из core-точек мы «разрастаем» кластер, добавляя все точки, достижимые по плотности.



4 Типы точек

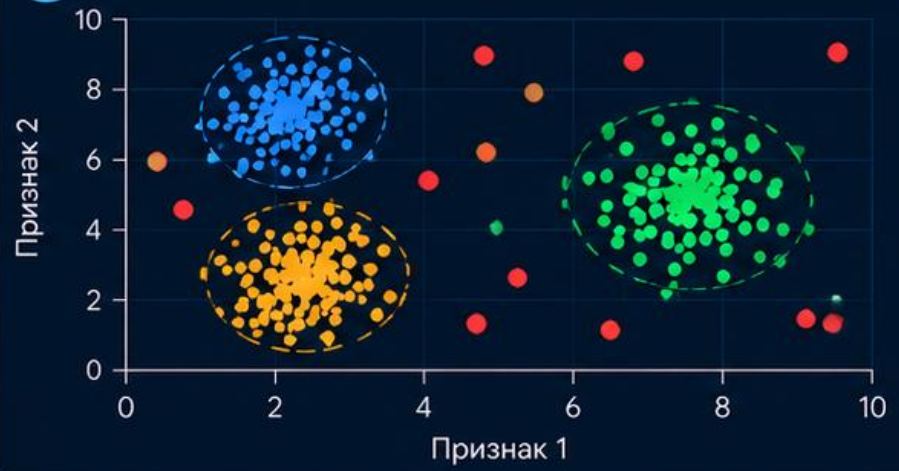


Core (ядро)
≥ MinPts соседей в радиусе ε. Принадлежит кластеру.

Border (граница)
< MinPts соседей, но находится в окрестности ядра. Принадлежит кластеру.

Noise (шум)
< MinPts соседей и не находится в окрестности ядра. Не принадлежит ни к одному кластеру.

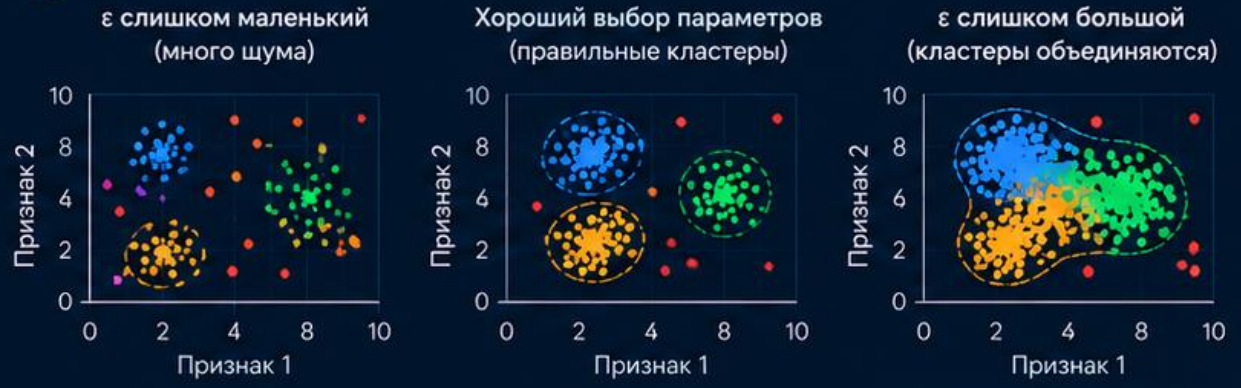
5 Пример работы DBSCAN



- Кластер 1
- Кластер 2
- Кластер 3
- Шум (noise)

Параметры:
ε = 0.8
MinPts = 5

6 Влияние параметров ε и MinPts



K-Means vs DBSCAN

Two Moons: Centroids vs Density

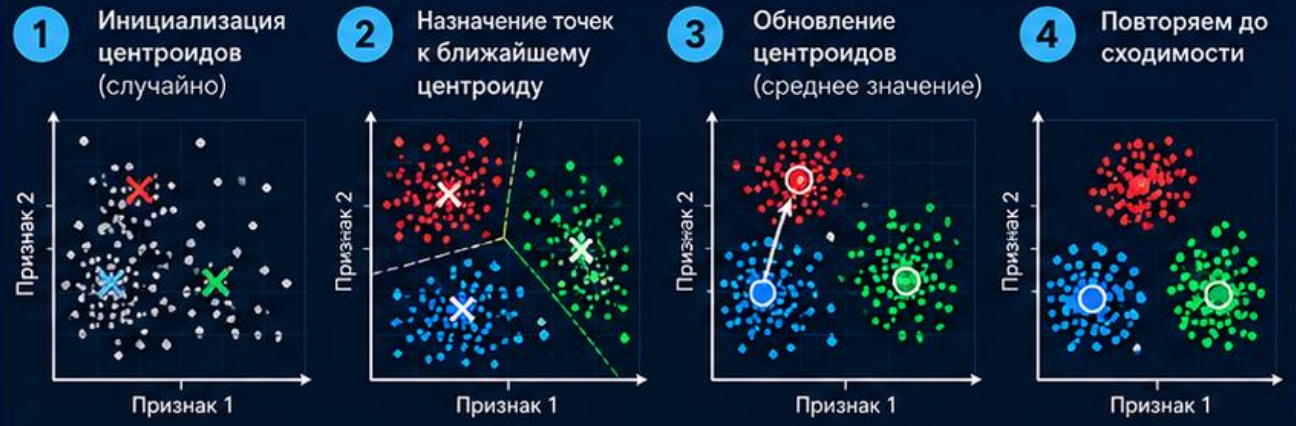
- K-Means relies on centers.
- For two moons, centroid-based partitions do not follow the shapes.
- DBSCAN can follow curved dense structures.



K-Means vs DBSCAN: сравнение двух алгоритмов кластеризации

K-Means Кластеризация на основе разделения (partition-based)

Идея: разделить данные на K кластеров, минимизируя расстояние между точками и центроидами кластеров.



Целевая функция (инерция, inertia):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- x_i — точка данных
- μ_k — центроид k-го кластера
- C_k — множество точек в кластере k
- $\|\cdot\|$ — евклидово расстояние
- J — сумма квадратов расстояний (инерция)

DBSCAN Кластеризация на основе плотности (density-based)

Идея: группировать точки, находящиеся в областях высокой плотности, и выделять точки шума, не принадлежащие ни к одному кластеру.



- **Ядровая точка** (core point) (≥ MinPts соседей)
- **Граничная точка** (border point) (< MinPts, но в радиусе ε от ядровой точки)
- **Шумовая точка** (noise point) (< MinPts и не в радиусе ε)

Параметры:

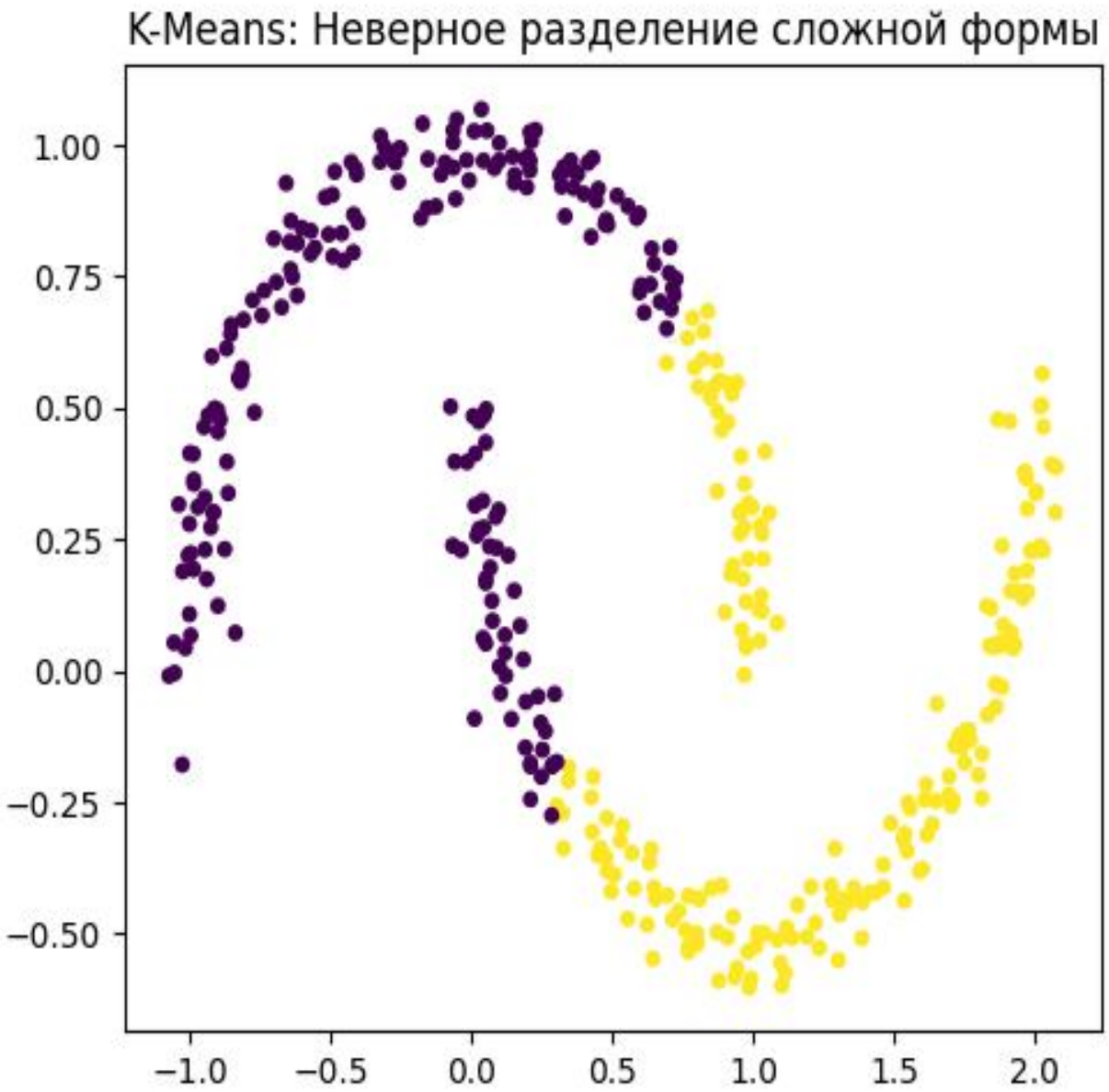
- ε (eps) — радиус окрестности
- MinPts — минимальное количество соседних точек для формирования плотной области

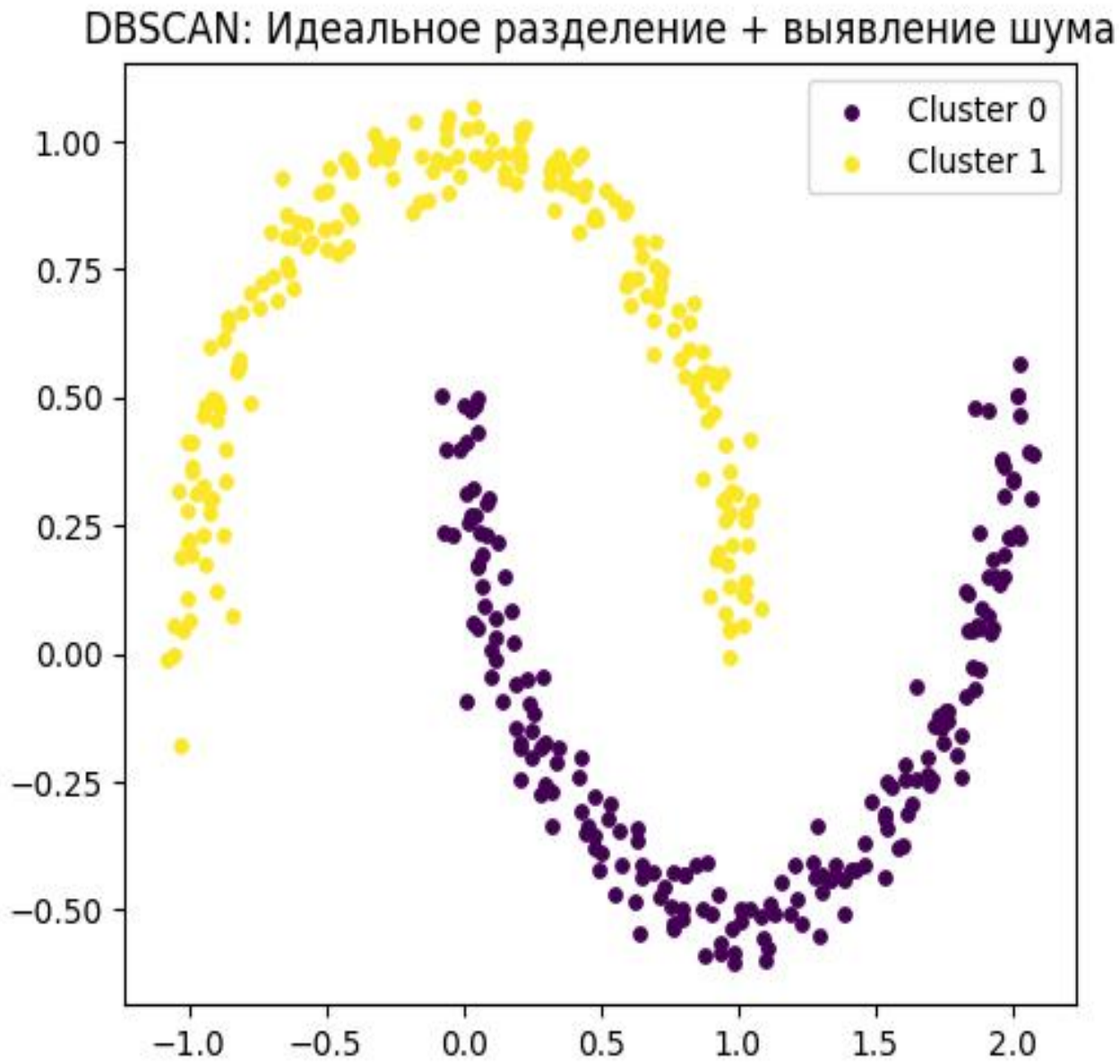
Пример окрестности:



Если в радиусе ε есть ≥ MinPts точек — точка считается ядерной.

Характеристика	K-Means	DBSCAN
Основная идея	Разделяет данные на K кластеров, минимизируя инерцию.	Группирует точки в области высокой плотности, выделяет шум.
Основные параметры	K (количество кластеров).	ε (радиус окрестности) и MinPts (минимальное количество точек).
Форма кластеров	Лучше работает для сферических, похожих по размеру кластеров.	Находит кластеры произвольной формы.
Плотность	Предполагает примерно одинаковую плотность в кластерах.	Может находить кластеры с разной плотностью.
Выбросы (шум)	Чувствителен к выбросам (они влияют на центроиды).	Явно выделяет шумовые точки.
Количество кластеров	K нужно задать заранее.	Определяется автоматически.
Масштабируемость	Быстрый и хорошо масштабируется на большие наборы данных.	Сложнее по вычислениям (зависит от реализации), хуже масштабируется для очень больших данных.
Типичные задачи	Сегментация клиентов, сжатие изображений, кластеризация документов (когда кластеры примерно сферические).	Геопространственные данные (GPS), детекция аномалий, анализ событий, кластеры сложной формы, данные с шумом.


K-Means: Неверное разделение сложной формы
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


DBSCAN: Идеальное разделение + выявление шума
Cluster 0
Cluster 1
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

DBSCAN in Python

Code and interpretation of label = -1

PYTHON

```
from sklearn.cluster import DBSCAN

db = DBSCAN(
    eps=0.35,
    min_samples=5
)

labels = db.fit_predict(X_scaled)

# -1 means noise
noise_count = (labels == -1).sum()
```

WHAT HAPPENS

DBSCAN does not use n_clusters.

eps sets the neighborhood radius.

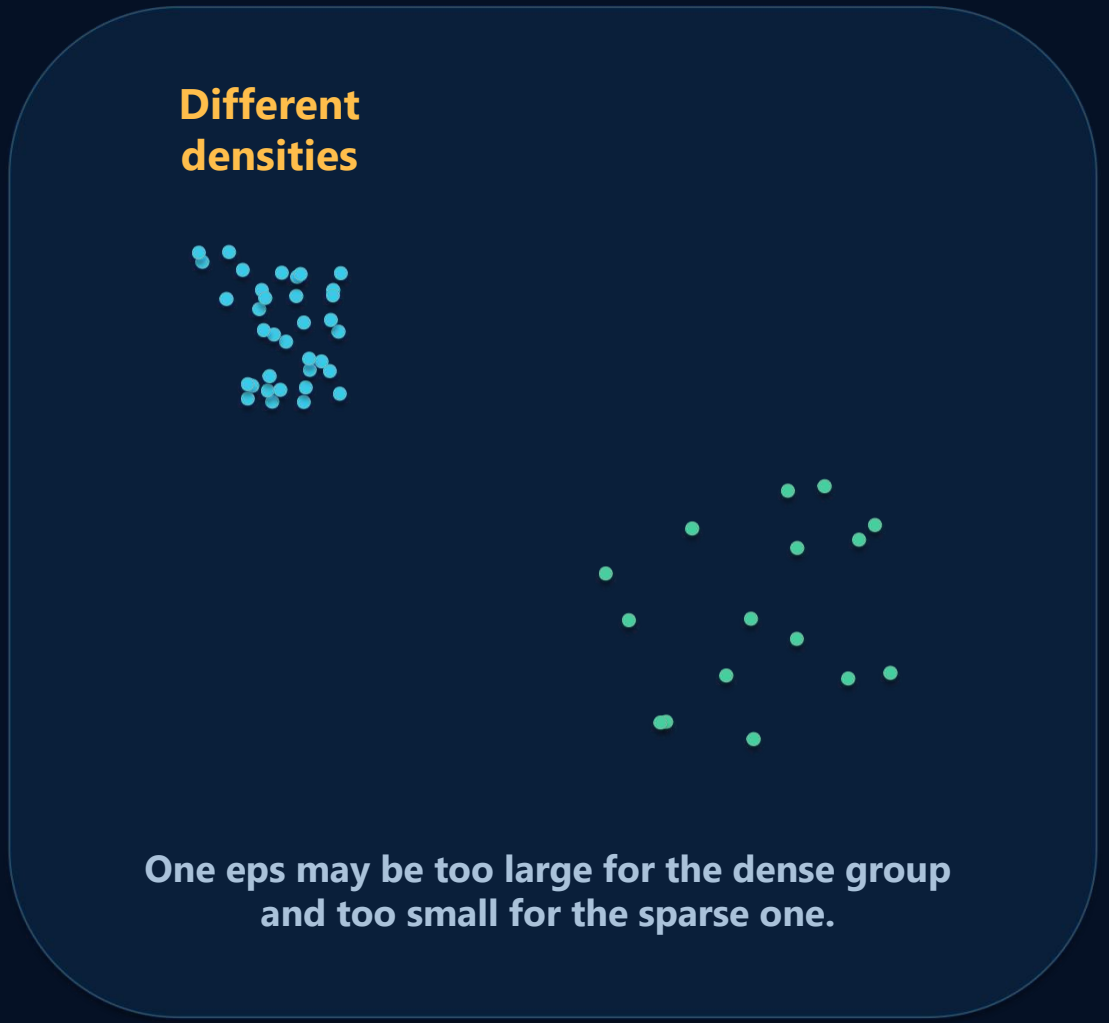
min_samples specifies the local density requirement.

label = -1 means a point was not assigned to a dense cluster.

When DBSCAN Performs Poorly

Different densities and difficult eps selection

- One eps works poorly for strongly varying densities.
- Distances become less informative in high dimensions.
- Parameters need tuning and visual checks.



Audience Question: Does Noise Mean Failure?

Why DBSCAN noise must not automatically be called a fault

AUDIENCE QUESTION

If DBSCAN labels an observation as noise, can we immediately declare it faulty?

ANSWER / DISCUSSION

No. Noise only means the observation is not part of a detected dense region. It could be a sensor error, a rare normal state, a new operating mode, or a genuine failure. Context must be checked.

NOISE ≠ FAILURE

Transition: Why Are Many Features a Problem?

From clustering to high-dimensional geometry

- So far, we have drawn 2D data.
- Real datasets may have 20, 100, or 1,000 features.
- The geometry of distances then changes.
- This leads to the curse of dimensionality.

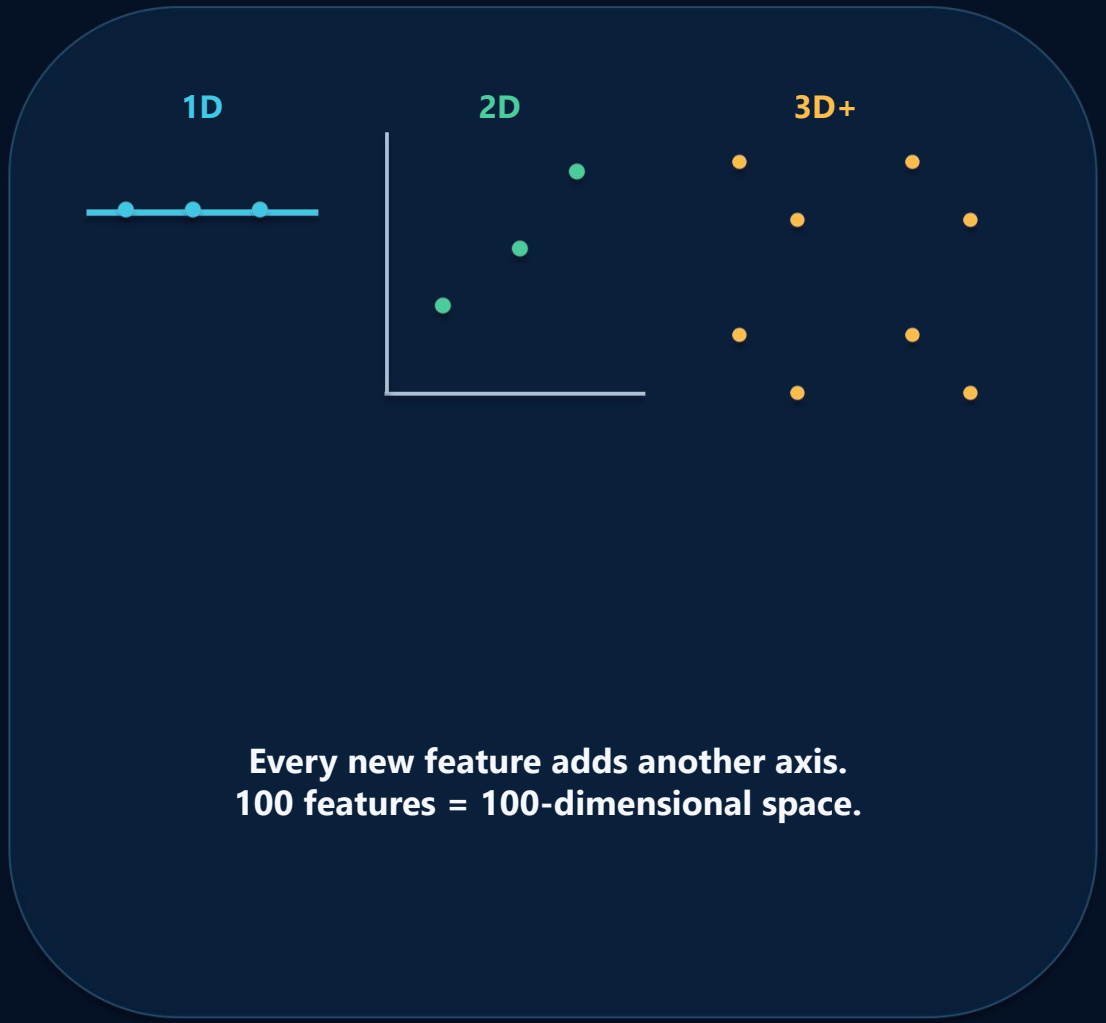


More dimensions mean more complex distance geometry.

What Is Dimensionality?

Every feature adds another axis

- 1 feature → a 1D line.
- 2 features → a 2D plane.
- 3 features → 3D space.
- p features → p-dimensional space.



The Curse of Dimensionality

Space volume grows exponentially

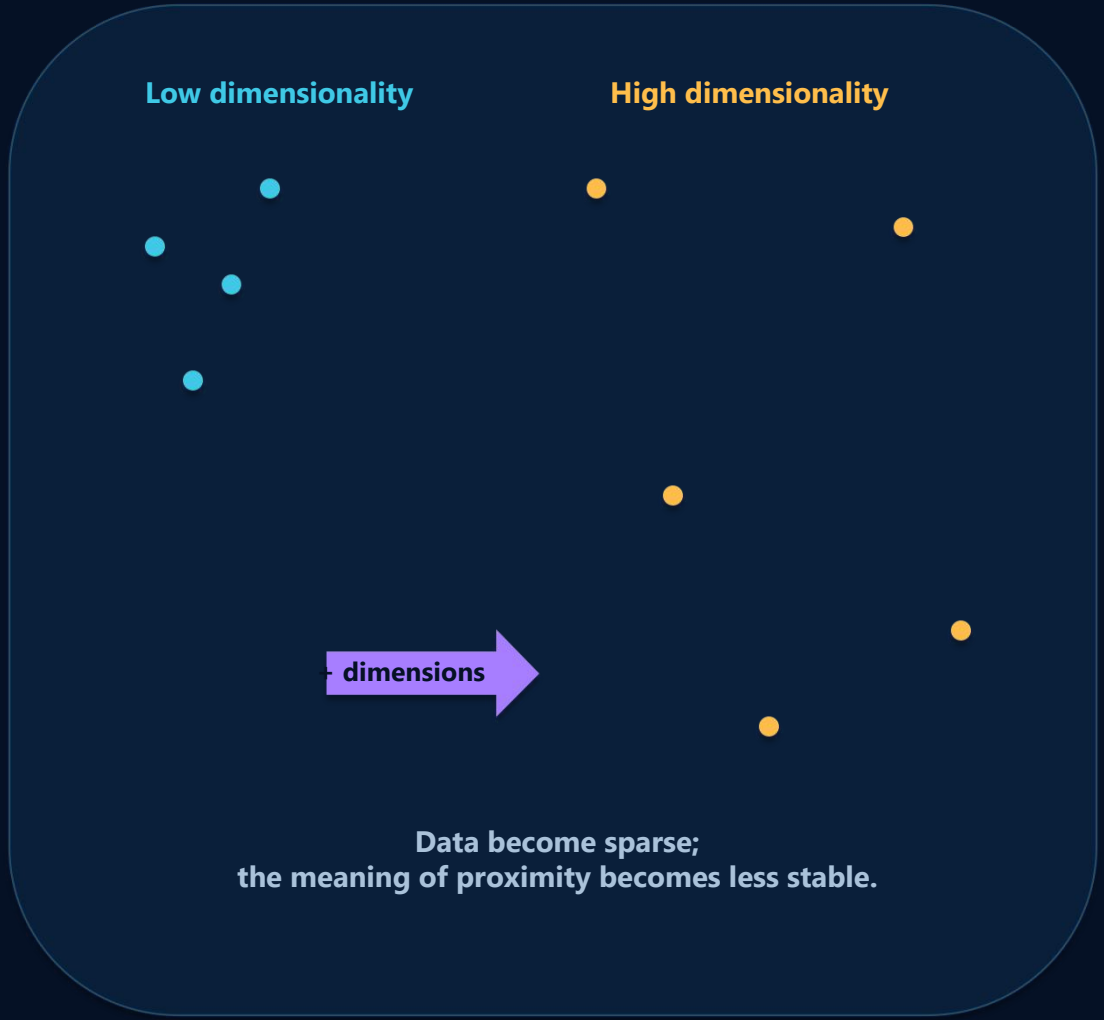
- 10 intervals in 1D → 10 regions.
- In 2D → 100.
- In 3D → 1,000.
- In 10D → 10^{10}.
- Space grows exponentially.



Why Distances Become Less Useful

Data become sparse in high dimensions

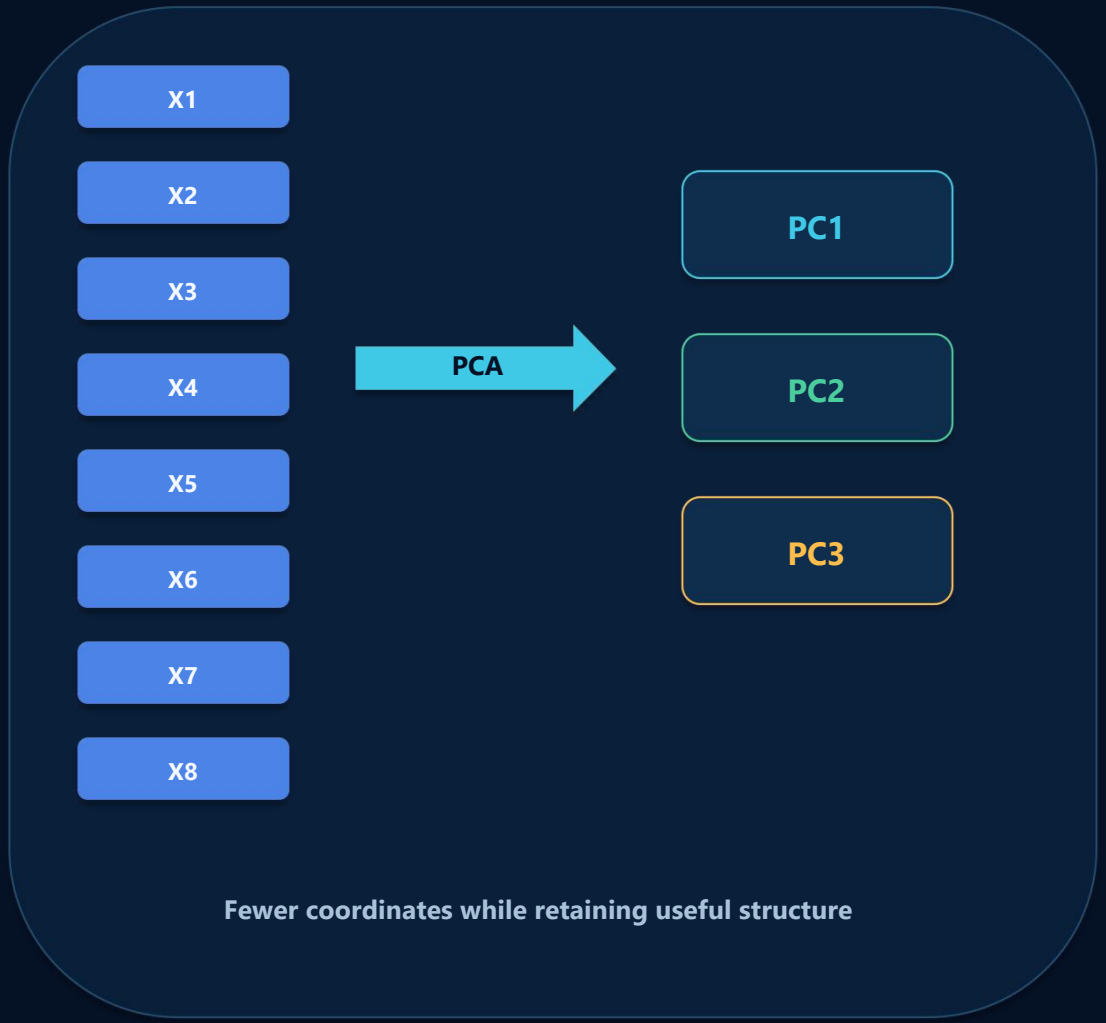
- With fixed N, observations become sparse.
- Distances between observations become less distinctive.
- Noisy features may dominate useful structure.



Why Reduce Dimensionality?

Compression, visualization, noise reduction, and redundancy

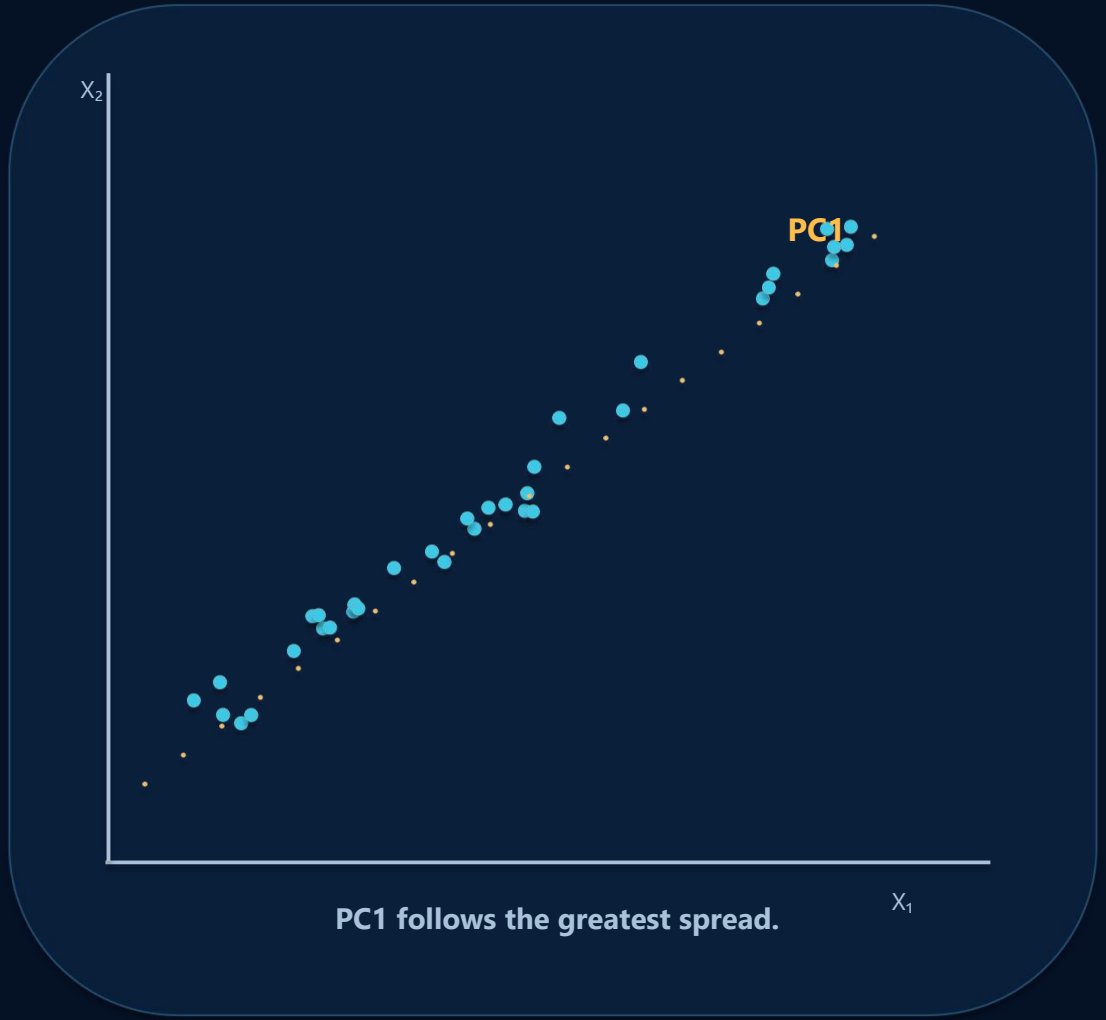
- Reduce computational complexity.
- Remove redundancy and some noise.
- Obtain a 2D/3D representation.
- Sometimes improve downstream workflows.



PCA: The Main Idea

Find directions of maximum variance

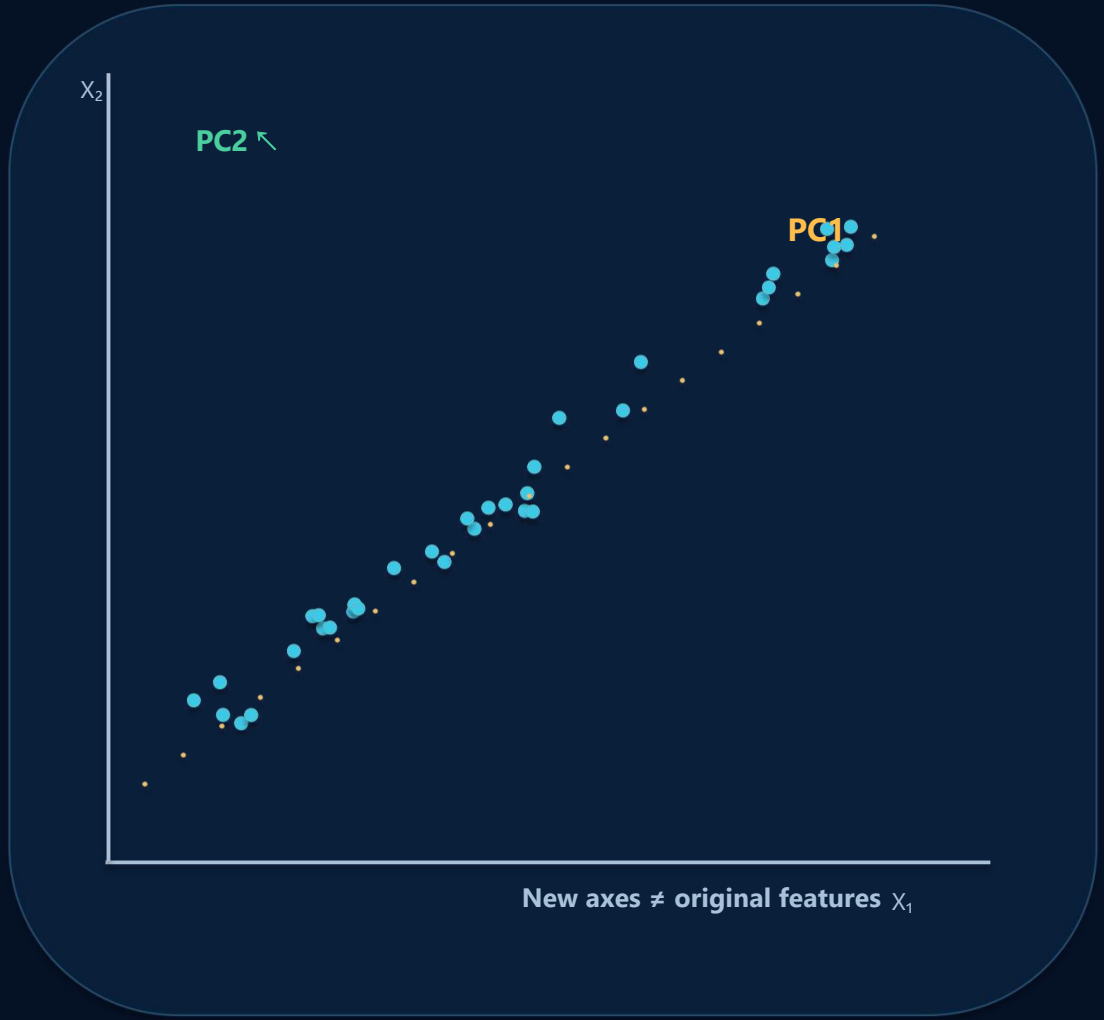
- PCA finds the direction of maximum variance.
- PC1 explains as much variability as possible.
- PC2 explains the maximum remaining variance and is orthogonal to PC1.



PCA as a Rotation of Coordinates

New axes PC1 and PC2 align with the data cloud

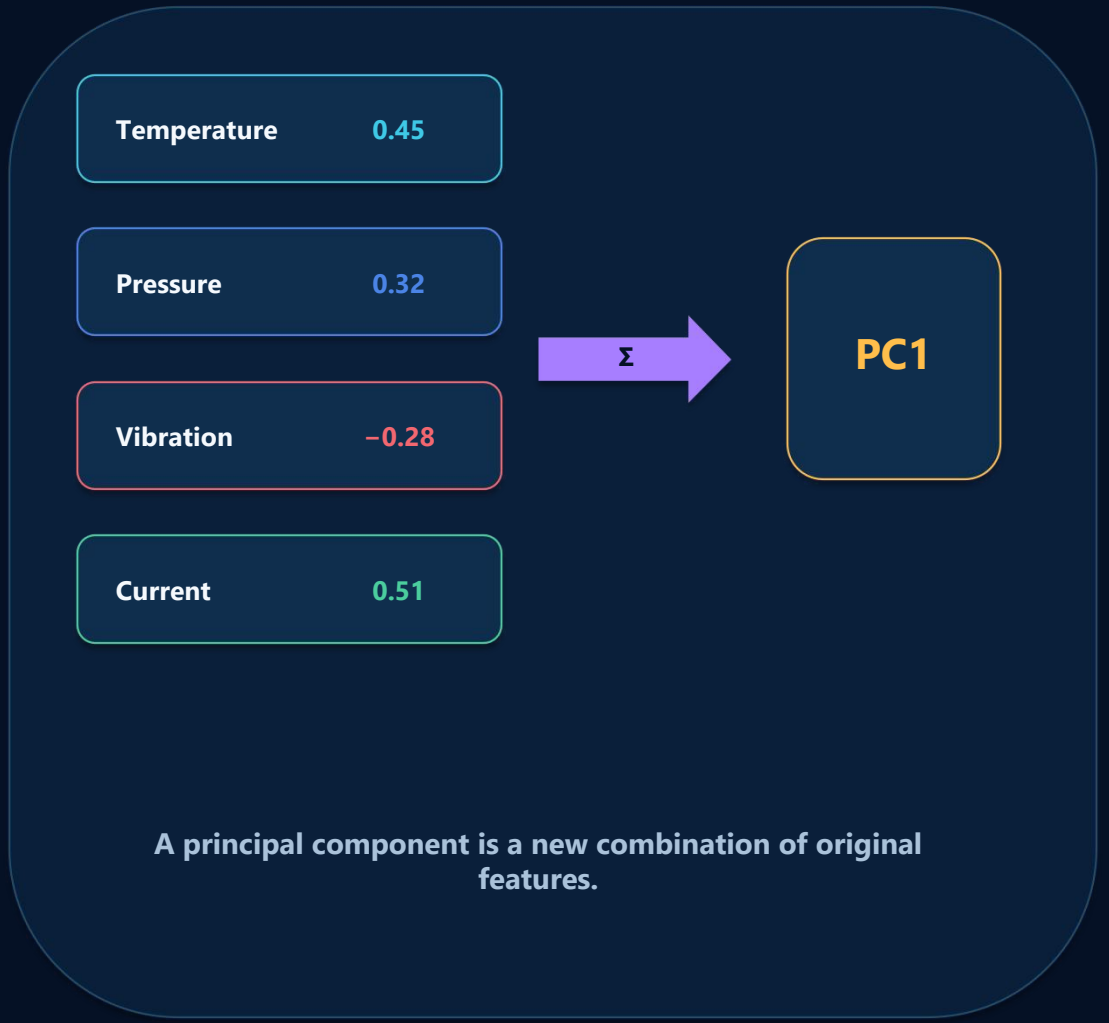
- Imagine an elongated diagonal cloud.
- Original X1/X2 axes are not optimal for describing it.
- PCA rotates the axes along the cloud.
- The weak direction may be discarded.



What Is a Principal Component?

A linear combination of original features

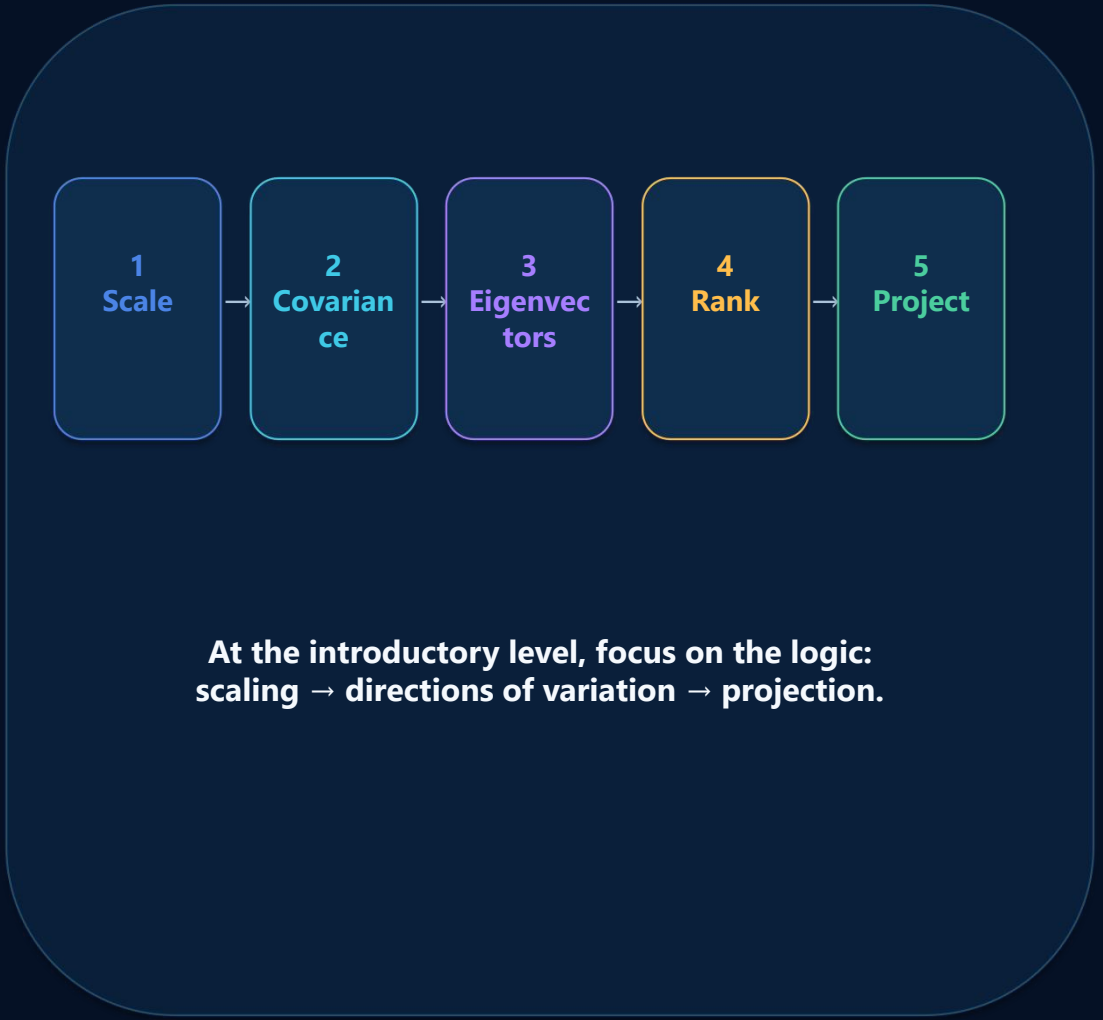
- PC1 = w1X1 + w2X2 + … + wpXp.
- It is a new synthetic feature.
- It is not identical to any original column.
- Therefore, interpreting a component can be more difficult.



PCA Step by Step

Scale → covariance → directions → projection

- 1. Scale the features.
- 2. Analyze their joint variability.
- 3. Find the principal directions.
- 4. Rank them by explained variance.
- 5. Project X into the new space.



PCA: главная идея

Найти новые оси (главные компоненты), по которым данные максимально вариативны, и спроецировать данные на эти оси, чтобы уменьшить размерность.

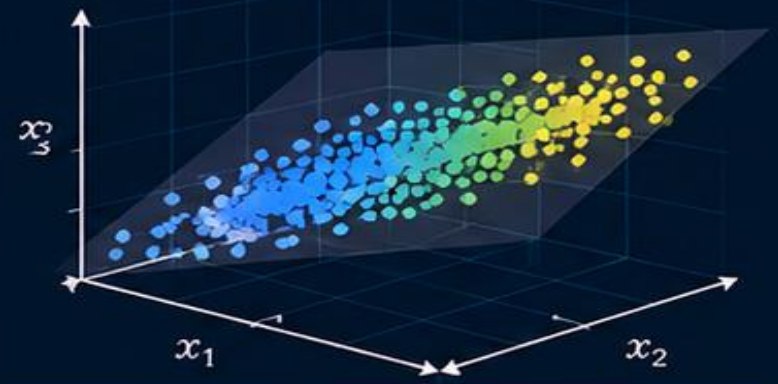
Математическая основа

1. Центрируем данные:
   $$X_c = X - \mu$$
2. Вычисляем ковариационную матрицу:
   $$\Sigma = \frac{1}{n-1} X_c^T X_c$$
3. Находим собственные значения и векторы:
   $$\Sigma v = \lambda v$$
4. Сортируем по убыванию λ и выбираем первые k векторов (главные компоненты).
5. Проекция данных:
   $$Z = X_c W_k$$

X — матрица данных $(n \times d)$
μ — вектор средних
Σ — ковариационная матрица $(d \times d)$
λ — собственные значения (показывают дисперсию)
v — собственные векторы (направления)
W_k — матрица первых k главных компонент
Z — данные в новом пространстве $(n \times k)$

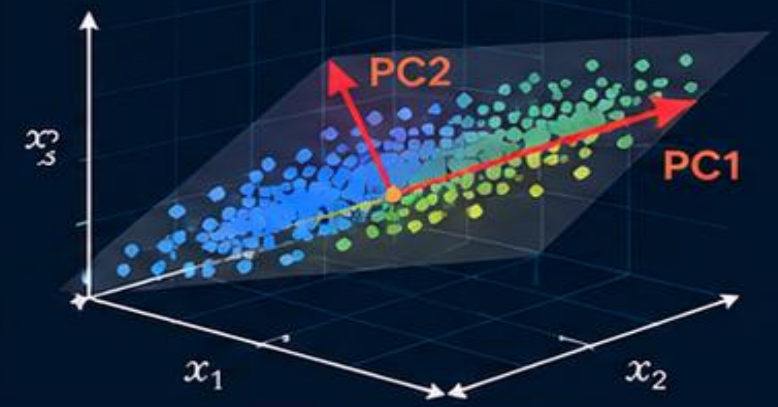
1 Исходные данные (высокая размерность)

Данные часто коррелированы и лежат около низкоразмерной структуры (например, плоскости).



2 Поиск главных компонент

Находим направления максимальной дисперсии (собственные векторы ковариационной матрицы).



3 Проекция на главные компоненты

Проецируем данные на первые k главных компонент (обычно $k = 2$) для визуализации или дальнейшего анализа.



4 Результат

Данные представлены в новом координатном пространстве, где первые компоненты сохраняют наибольшую часть дисперсии.



Дисперсия, объясняемая компонентами



Ключевые выводы

PCA не изменяет данные, а линейно преобразует их.

- Первая главная компонента содержит наибольшую дисперсию.
- Главные компоненты ортогональны (некоррелированы).
- Позволяет уменьшить размерность при минимальной потере информации.
- Широко используется для визуализации, сжатия данных и подготовки признаков для обучения моделей.

PCA in Python

Two components and the n_components=0.95 option

PYTHON

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)

pca2 = PCA(n_components=2)
X_2d = pca2.fit_transform(X_scaled)

pca95 = PCA(n_components=0.95)
X_compact = pca95.fit_transform(X_scaled)
```

WHAT HAPPENS

n_components=2 is useful for 2D visualization.

n_components=0.95 retains the minimum number of components explaining approximately 95% of the variance.

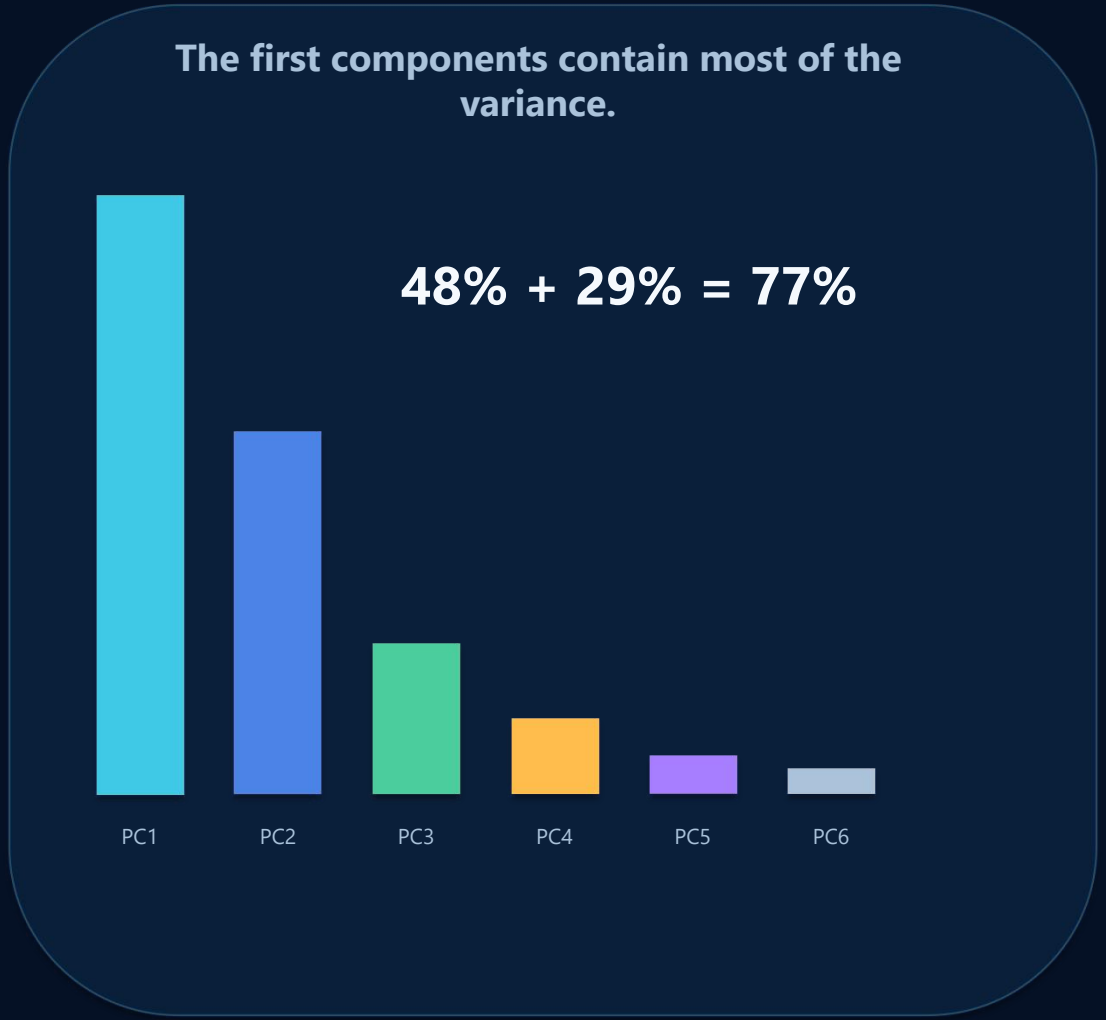
Usually scale features before PCA.

50 features → PCA → 2D / 95%

Explained Variance

How Much Variability Does Each Component Retain?

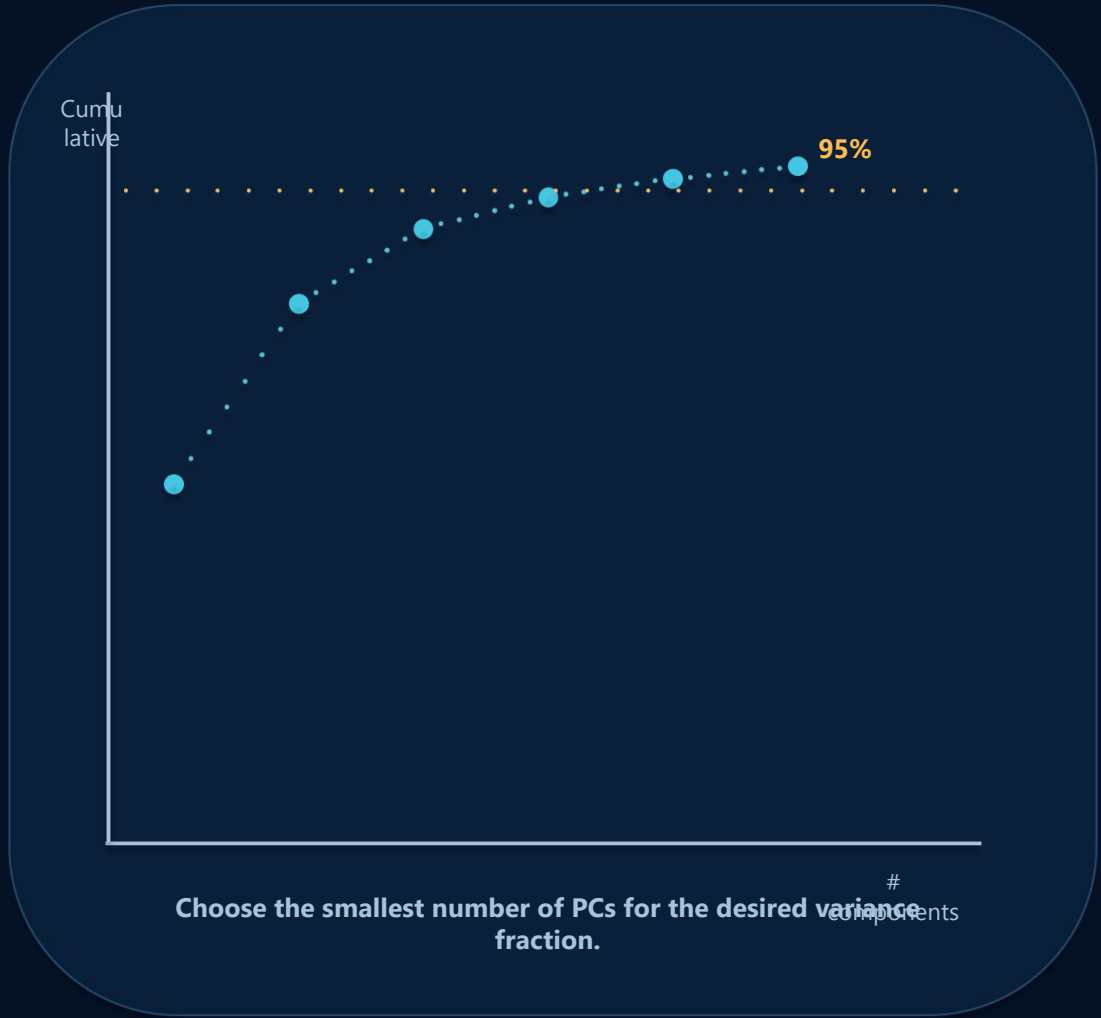
- Example: PC1=48%, PC2=29%, PC3=12%.
- The first two components retain 77%.
- The first three retain 89%.
- Explained variance shows the cost of dimensionality reduction.



How Many Components Should We Keep?

Cumulative explained variance: 90–95% as a guideline

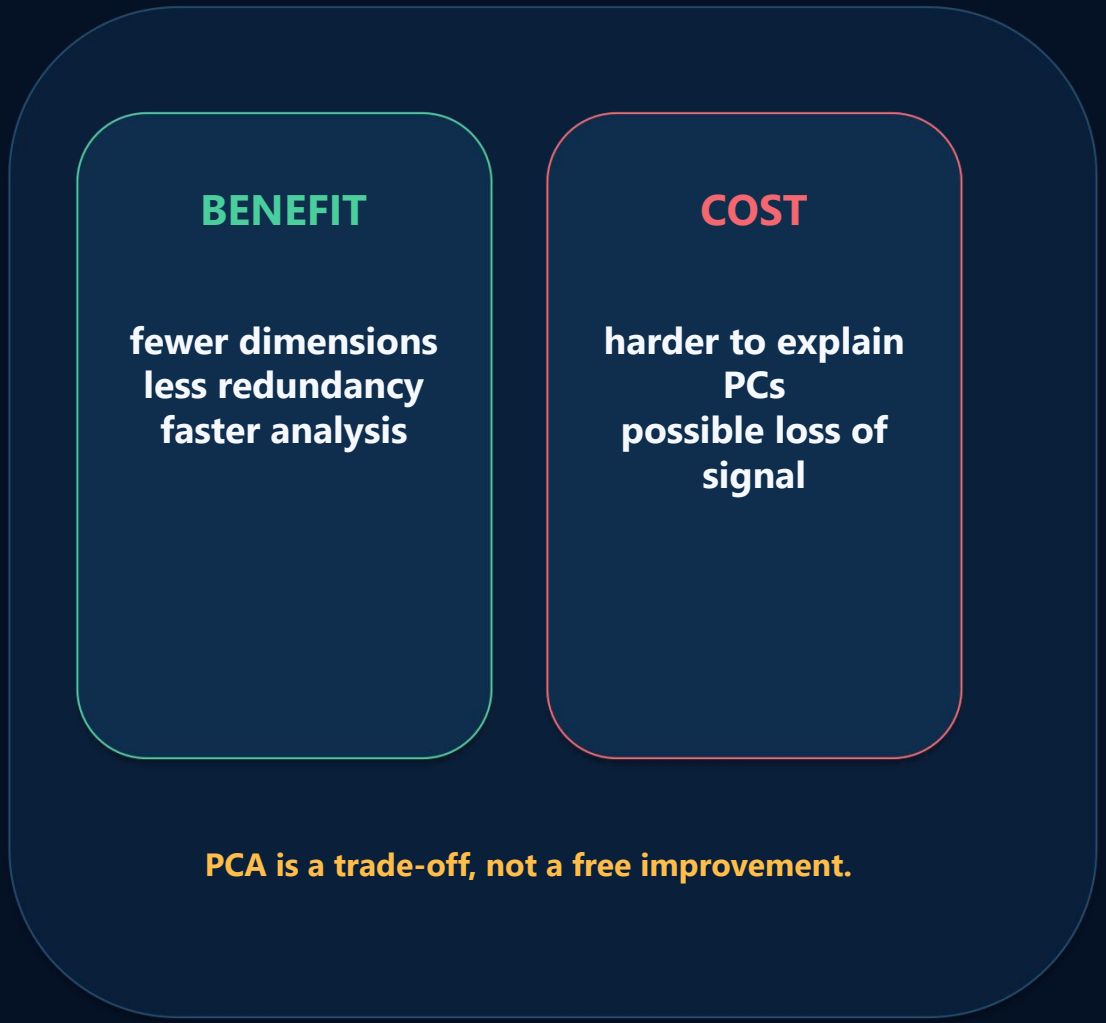
- Plot cumulative explained variance.
- Find the number of components for the desired information level.
- For example, 90% or 95% is a guideline, not a rule.
- You can use PCA(n_components=0.95).



PCA: Benefits and Trade-offs

Compactness versus interpretability

- Benefits: fewer dimensions, less redundancy, easier visualization.
- Trade-off: components are harder to interpret.
- Excessive compression may remove useful signals.



Data Leakage and PCA

Fit PCA only on the training fold if a supervised model follows

- If PCA precedes a supervised model, do not fit it on the full dataset.
- Otherwise test-fold information influences the projection.
- Correct: StandardScaler → PCA → Model inside Pipeline/CV.



Why Use t-SNE?

A nonlinear map of local neighborhoods

- t-SNE creates a 2D/3D map of local neighborhoods.
- It tries to keep similar observations close.
- Useful for exploring embeddings and complex data.
- Axes are not directly interpretable.

PCA is linear and cannot unfold a twisted spiral. Nonlinear approaches such as t-SNE and UMAP can help visualize such structure.

IMPORTANT: Do not treat t-SNE embeddings as stable, globally distance-preserving features for downstream modeling.

Its main purpose here is exploratory visualization.



t-SNE tries to keep similar observations nearby.

How NOT to Interpret t-SNE

Beautiful islands are not proof of real classes

- Distances between far-apart islands should not be read literally.
- Visual cluster sizes may be distorted.
- Different parameters may change the picture.
- A beautiful plot ≠ proof of classes.



UMAP

Fast Nonlinear Visualization of Complex Structure

- UMAP produces a nonlinear low-dimensional representation.
- Often faster than t-SNE on large datasets.
- Useful for exploratory visualization.
- Interpret global geometry with caution.



UMAP: local structure and often faster computation

UMAP: основная идея

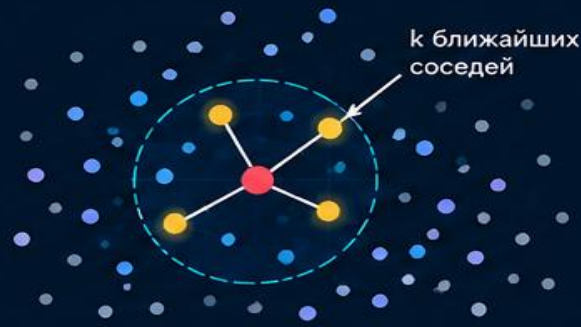
UMAP (Uniform Manifold Approximation and Projection) — метод нелинейного снижения размерности, который сохраняет локальную структуру данных, используя понятия многообразия и теорию графов.

Интуитивно:

UMAP предполагает, что данные лежат на низкоразмерном многообразии, встроенном в пространство высокой размерности, и пытается восстановить это многообразие в низкоразмерном пространстве, сохраняя отношения между соседними точками.

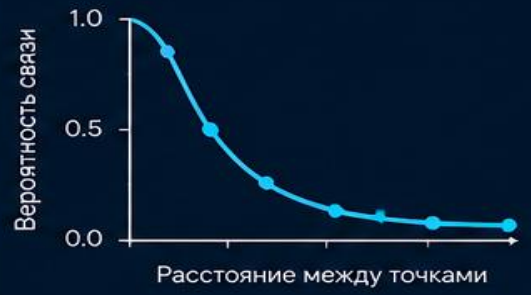
1 Построение локальных соседств в исходном пространстве

Для каждой точки находим k ближайших соседей и строим взвешенный граф. Вероятность связи зависит от расстояния: ближайшие точки имеют большую вероятность быть соседями.



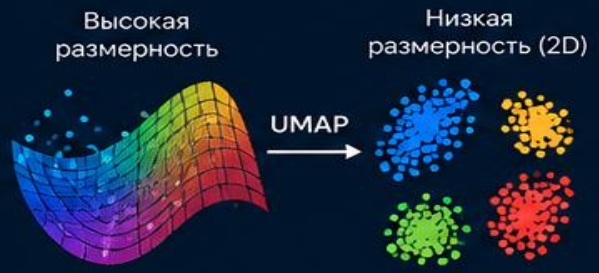
2 Преобразование расстояний в вероятности

Расстояния между точками преобразуются в вероятности с помощью сглаживающей функции, которая адаптируется к локальной плотности (ближе точки → больше вероятность).



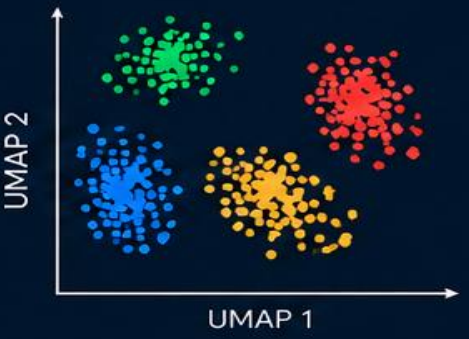
3 Оптимизация расположения в низкой размерности

Ищется расположение точек в низкоразмерном пространстве (обычно 2D или 3D), которое сохраняет вероятности соседства, минимизируя расхождение между исходным и новым распределением соседей (кросс-энтропия).

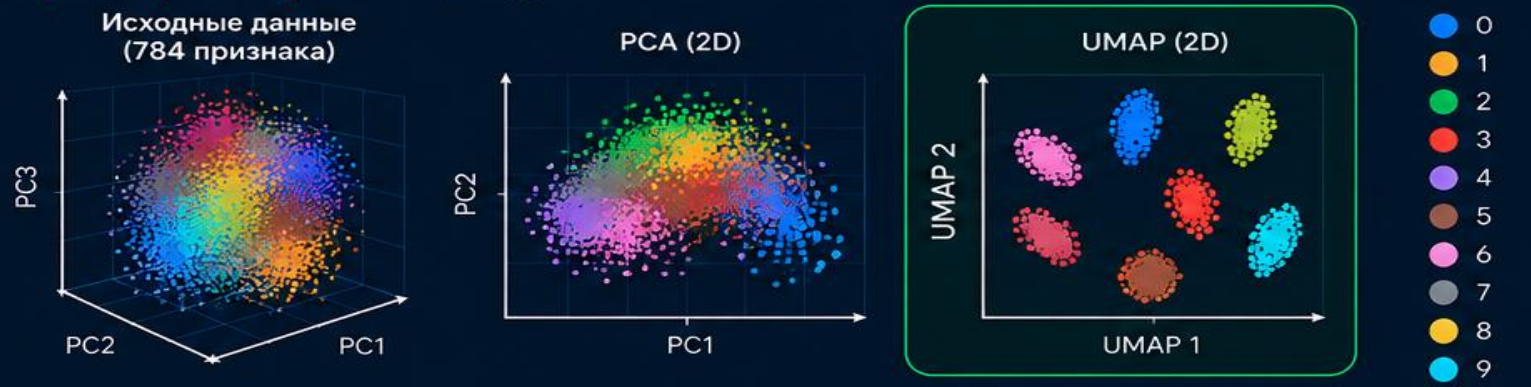


4 Результат

Точки отображаются в 2D/3D так, что локальная структура (соседи) хорошо сохраняется, позволяя визуализировать кластеры, структуру данных и нелинейные отношения.



Пример: визуализация данных MNIST



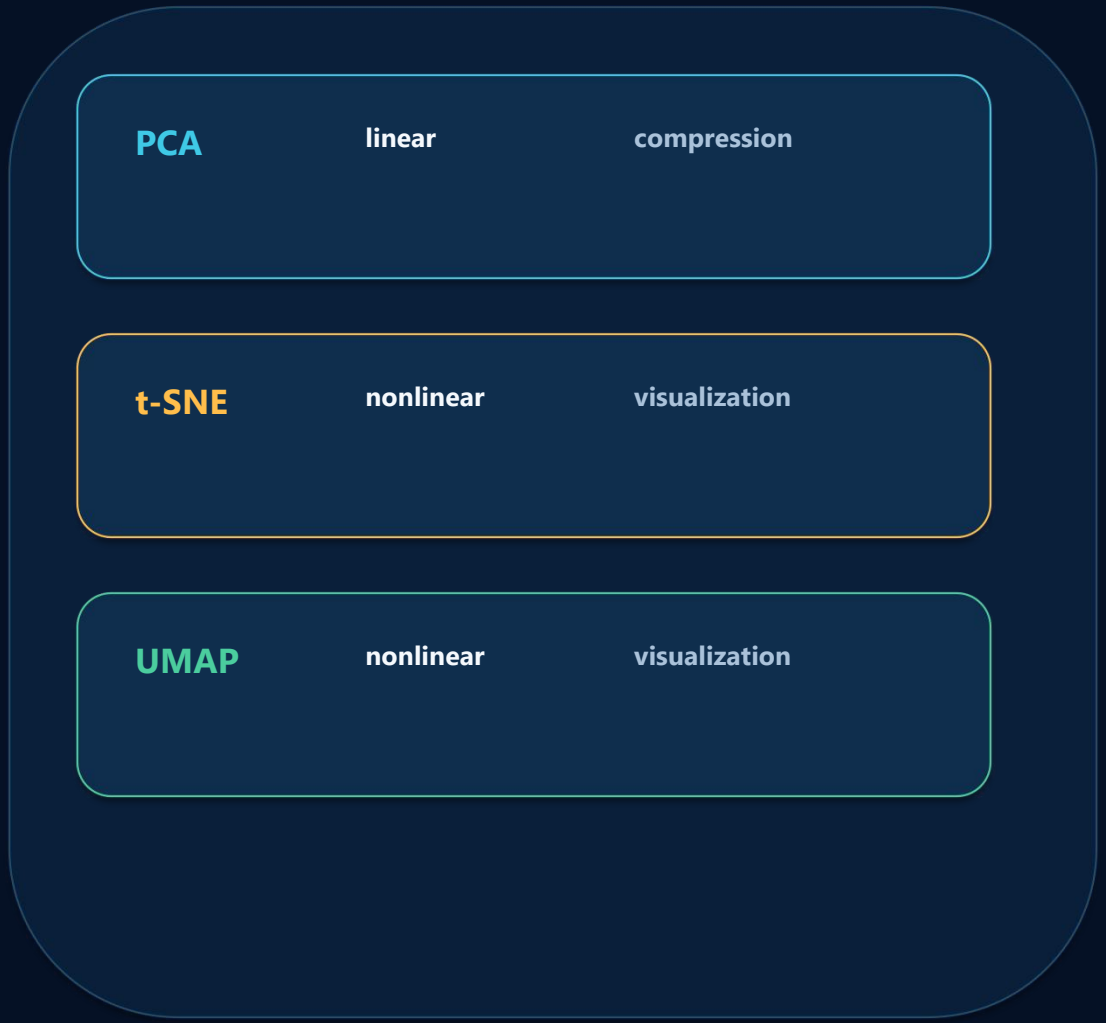
Ключевые особенности UMAP

- Сохраняет локальную структуру данных (лучше, чем линейные методы).
- Хорошо работает с нелинейными структурами (на многообразиях).
- Быстрее, чем t-SNE, особенно для больших данных.
- Позволяет визуализировать кластеры, структуру и аномалии.
- Имеет важные гиперпараметры: n_neighbors (размер локального соседства) и min_dist (минимальное расстояние между точками в новом пространстве).

PCA vs t-SNE vs UMAP

Choose the Method Based on the Goal

- PCA: linear, fast, useful for compression.
- t-SNE: exploratory visualization of local neighborhoods.
- UMAP: nonlinear visualization, often scales better.
- Choose based on the task, not the beauty of the plot.



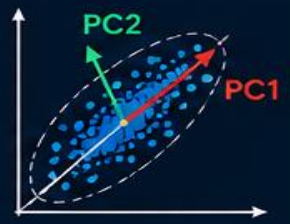
PCA vs t-SNE vs UMAP

Три популярных метода снижения размерности: разные идеи, цели и свойства

PCA Линейное проецирование

Основная идея:

Найти новые ортогональные оси (главные компоненты), по которым данные имеют максимальную дисперсию, и спроецировать данные на эти оси.



Ключевые свойства:

- Линейный метод
- Максимизирует дисперсию
- Быстрый и масштабируемый
- Хорошо интерпретируется (компоненты — линейные комбинации исходных признаков)

t-SNE Сохранение локальной структуры

Основная идея:

Преобразовать расстояния между точками в вероятности схожести и найти отображение в низкоразмерное пространство, которое сохраняет локальные соседства (близкие точки остаются близкими).



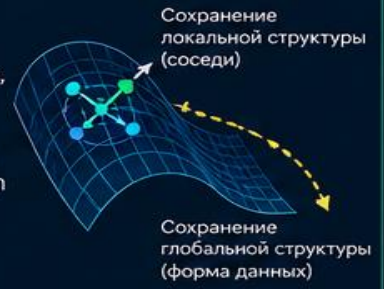
Ключевые свойства:

- Нелинейный метод
- Хорошо сохраняет локальную структуру (кластеры)
- Подходит для визуализации
- Медленный на больших данных
- Чувствителен к выбору параметров (perplexity, learning rate)

UMAP Сохранение локальной и глобальной структуры

Основная идея:

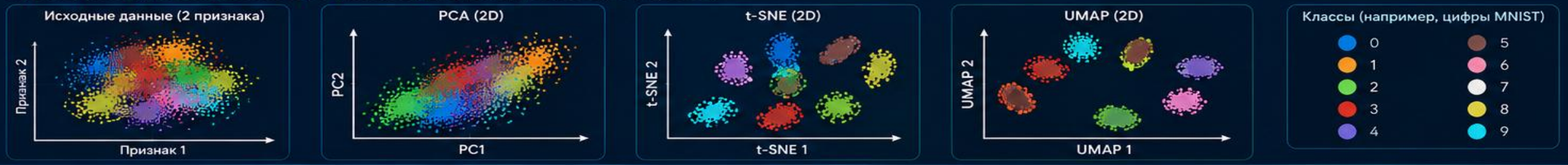
Построить граф ближайших соседей в исходном пространстве, затем найти низкоразмерное отображение, которое сохраняет структуру этого графа, используя теорию многообразий (Riemannian geometry) и оптимизацию кросс-энтропии.



Ключевые свойства:

- Нелинейный метод (быстрее t-SNE)
- Сохраняет и локальную, и глобальную структуру
- Хорошо работает на больших данных
- Устойчивее к выбору параметров
- Позволяет применять к новым данным (transform)

Пример: визуализация набора данных (например, MNIST, 10 классов)



Характеристика	PCA	t-SNE	UMAP
Тип метода	Линейный	Нелинейный	Нелинейный
Основная цель	Максимизировать дисперсию	Сохранить локальные соседства	Сохранить локальную и глобальную структуру
Что сохраняет	Глобальную вариативность (линейную)	Локальную структуру	Локальную и глобальную структуру
Скорость	Очень быстрый	Медленный	Быстрый (обычно быстрее t-SNE)
Масштабируемость	Отличная (большие данные)	Плохая (сложно для больших данных)	Хорошая (подходит для больших данных)
Интерпретируемость	Высокая (главные компоненты)	Низкая	Средняя
Чувствительность к параметрам	Низкая	Высокая (perplexity и др.)	Ниже, чем у t-SNE
Новые данные (transformt)	Да	Ограниченно (нет явного преобразования)	Да
Типичные применения	Предобработка, сжатие данных, удаление шумов, линейный анализ	Визуализация кластеров в малых и средних наборах данных	Визуализация, исследование структуры данных, большие наборы, подготовка признаков

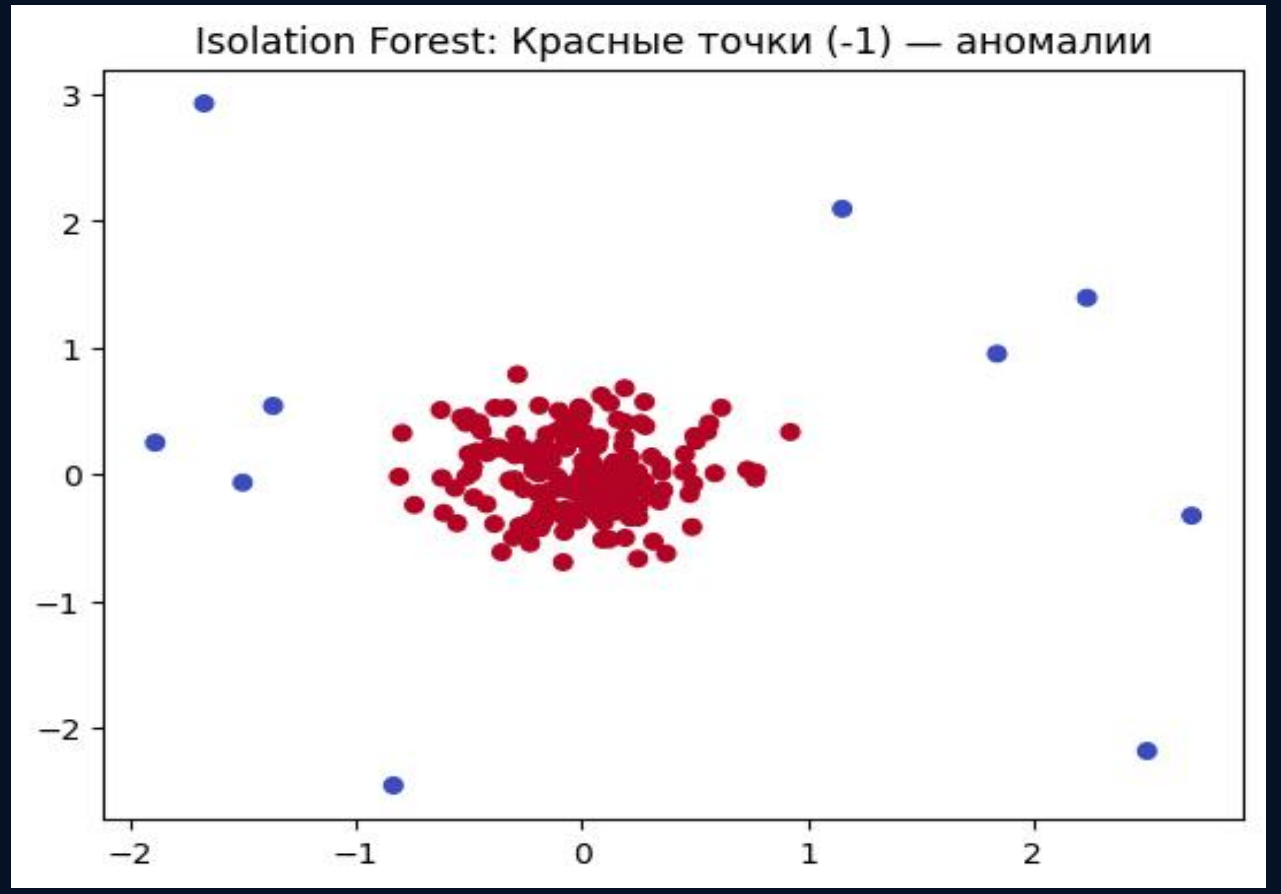
Ключевые выводы

- PCA — простой, быстрый и интерпретируемый линейный метод.
- t-SNE — отлично показывает локальные кластеры, но медленный и искажает глобальные расстояния.
- UMAP — сочетает преимущества: сохраняет локальные и глобальные структуры, работает быстрее t-SNE и подходит для больших данных.

What Is an Anomaly?

A rare observation or unusual combination of features

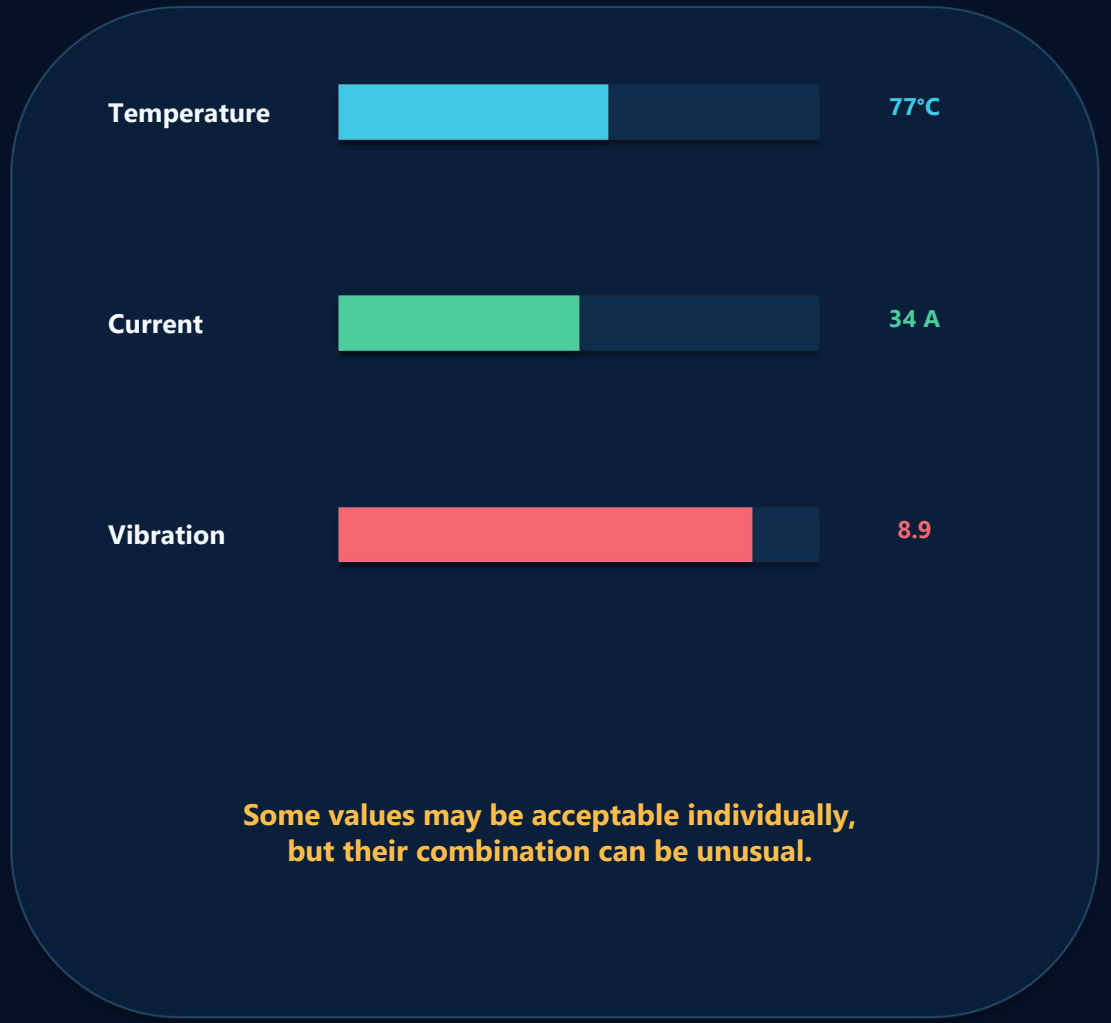
- An anomaly differs from the main data structure.
- It may be fraud, sensor error, failure, or a rare normal operating mode.
- The goal is to identify candidates for investigation.



Why Is a Simple Threshold Not Enough?

Individually normal readings can form an abnormal combination

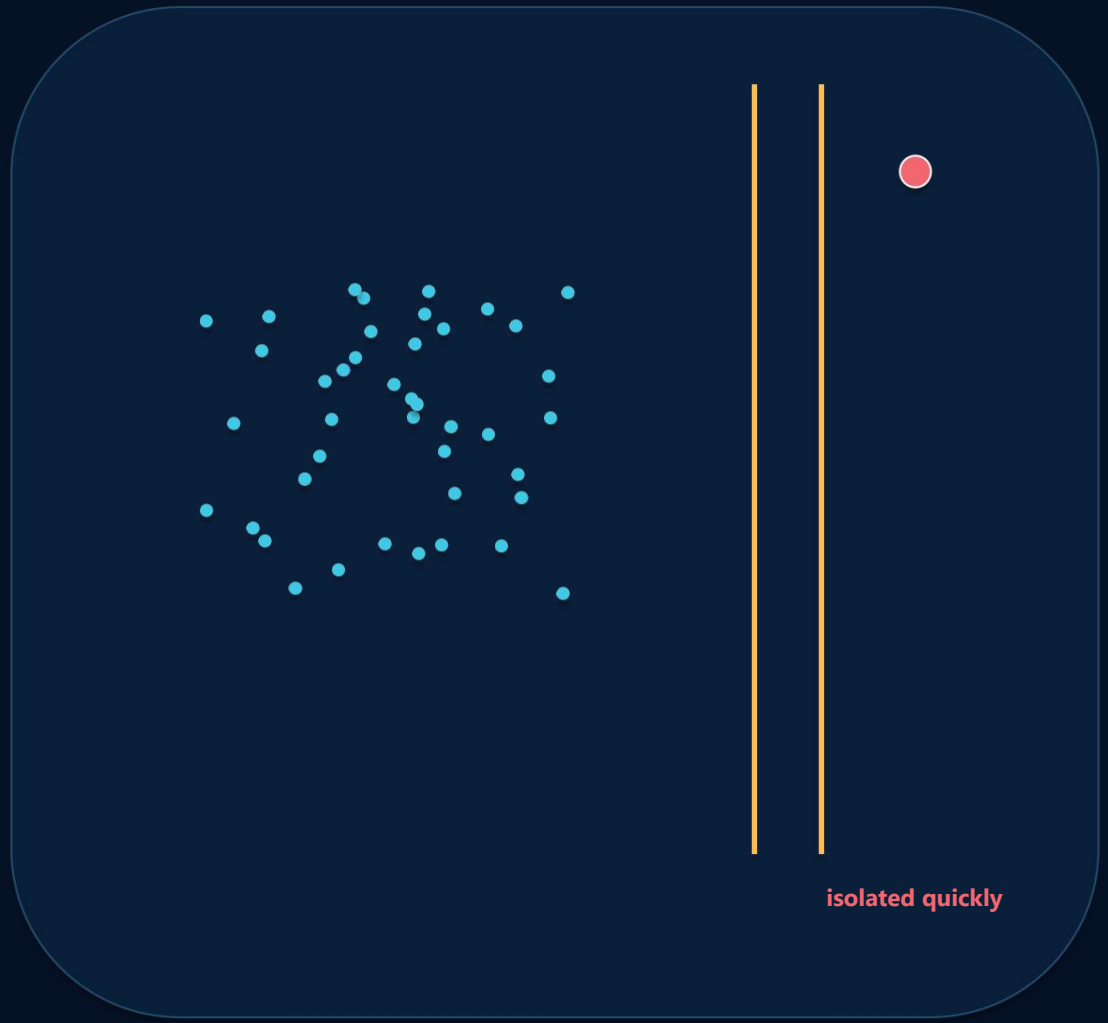
- A threshold works well for one clearly understood signal.
- A multivariate anomaly may consist of individually normal values.
- The unusual combination is what matters.



Isolation Forest: Intuition

Anomalies are easier to isolate with random splits

- Isolation Forest does not model normality directly.
- It attempts to isolate each point quickly.
- Rare points are usually easier to isolate.
- Shorter path → more anomalous observation.



Пошаговый алгоритм (псевдокод)

1. Для $t = 1 \ldots T$ (число деревьев):
   a) случайно выбрать подмножество данных;
   b) построить изолирующее дерево с случайными разделениями.
2. Для каждой точки x вычислить среднюю длину пути $E[h(x)]$.
3. Нормализовать длину пути с помощью $c(n)$.
4. Вычислить оценку аномальности $s(x)$.
5. Пометить точки с $s(x) > threshold$ как аномалии.

Ключевые гиперпараметры

Параметр	Обозначение	Рекомендации	Описание
Число деревьев	n_estimators	100–300	Больше деревьев → стабильнее результат.
Размер подвыборки	max_samples	256 (или auto)	Случайное подмножество данных для каждого дерева.
Максимальная глубина	max_depth	$\lceil \log_2(n) \rceil$	Обычно устанавливается автоматически.
Порог аномальности	contamination	0.01 – 0.1	Ожидаемая доля аномалий (если неизвестна, можно не задавать).

1 Исходные данные

У нас есть набор данных с нормальными точками и аномалиями.



2 Случайный выбор признака и порога

На каждом узле случайно выбирается признак и случайное значение (между min и max), которое делит данные на две части.



3 Рекурсивное разделение

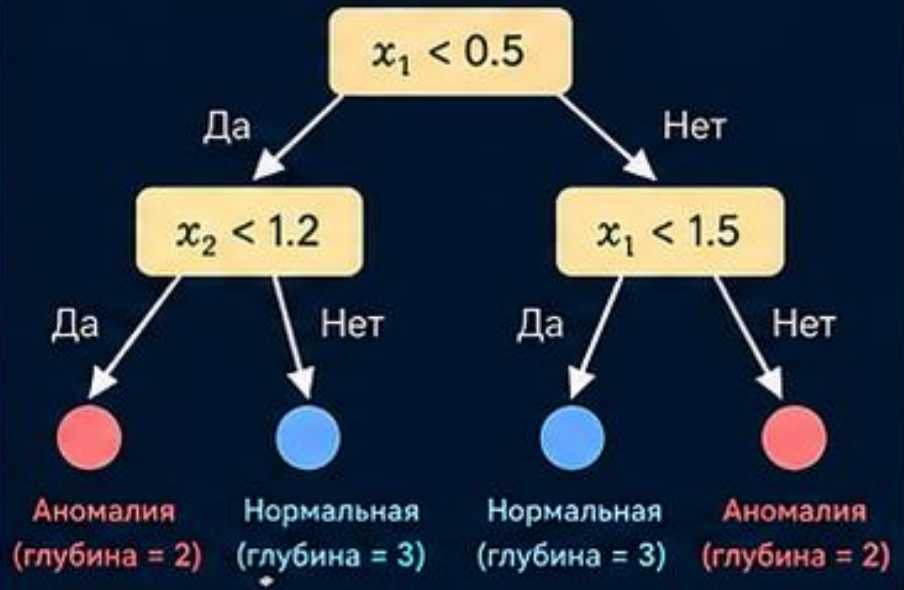
Для каждой части снова случайно выбирается признак и порог.
Процесс повторяется, пока:

- узел содержит 1 точку (точка изолирована),
- или достигнута максимальная глубина дерева.



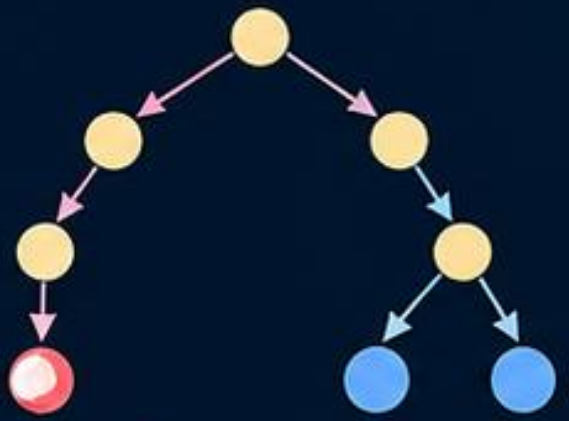
4 Пример построенного дерева

Дерево строится случайными разделениями. Аномальные точки оказываются ближе к корню (меньшая длина пути).



5 Длима пути

Для каждой точки считаем длину пути $h(x)$ — сколько разделений нужно, чтобы изоьировать точку.



$h(x) = 2$ (аномалия)

$h(x) = 5$ (нормальная)

6 Средняя длина пути

Строим много деревьев (например, 100–300). Для каждой точки вычисляем среднюю длину пути $h(x)$ по всем деревьям.

Точка	Дерево 1	Дерево 2	Дерево 3	...	Среднее $h(x)$
A (норм.)	5	6	5	...	5.3
B (норм.)	6	5	6	...	5.7
C (аном.)	2	2	3	...	2.3

7 Нормализация и оценка аномальности

Используем ожидаемую длину пути для случайного дерева с n точками:

$$c(n) = 2H(n-1) - 2\frac{(n-1)}{n}$$
$$\approx 2(ln(n-1) + \gamma) - 2\frac{(n-1)}{n}$$

где $H(n)$ — гармоническое число, $\gamma \approx 0.577$.

Оценка аномальности (чем ближе к 1, тем более аномальная точка):

$$s(x) = 2^{-\frac{E[h(x)]/c(n)}{}} \qquad 0 \le s(x) \le 1$$

Плюсы и минусы

Плюсы

- Простая и быстрая модель.
- Хорошо работает с большими данными.
- Не требует предположений о распределении.
- Эффективно выявляет редкие аномалии.
- Устойчив к выбросам и шуму.

Минусы

- Меньшая точность при сложных структурах аномалий.
- Может плохо работать на очень высоких размерностях (по сравнению с другими методами).
- Требует настройки параметра contamination (если нужен точный порог).

Isolation Forest in Python

contamination, labels, and decision_function

PYTHON

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(
    n_estimators=200,
    contamination=0.05,
    random_state=42
)

labels = iso.fit_predict(X_scaled)
scores = iso.decision_function(X_scaled)

# labels: 1 normal, -1 anomaly
```

WHAT HAPPENS

contamination affects the classification threshold.

labels are convenient for quick filtering.

decision_function provides scores to rank the most suspicious rows.

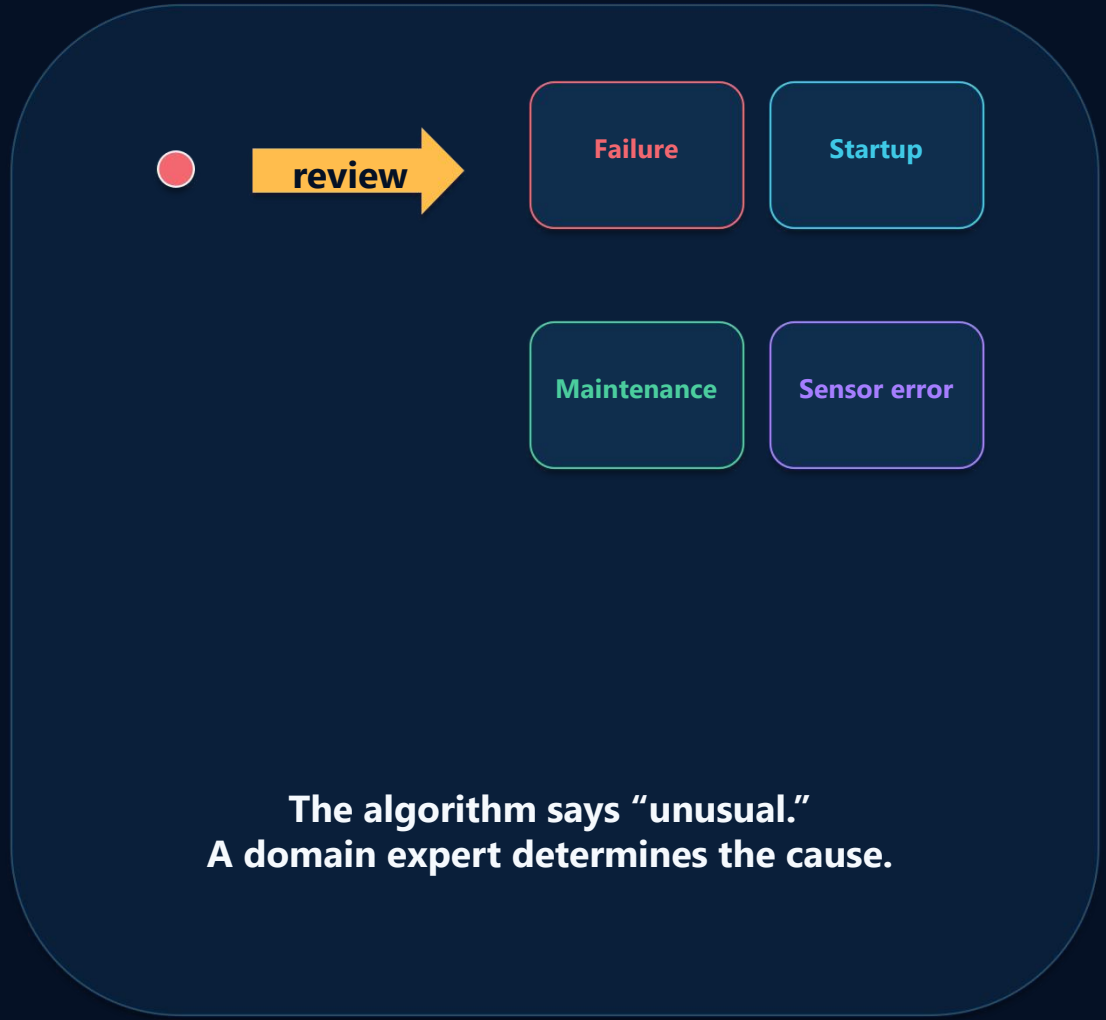
An expert must confirm the final conclusion.

1 = normal • −1 = anomaly
score → ranking

Anomaly ≠ Failure

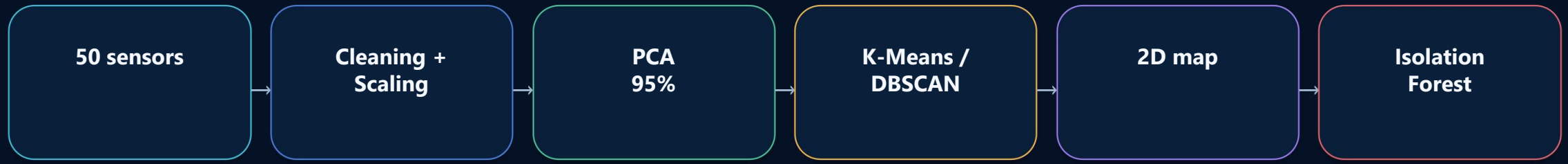
The algorithm identifies candidates for expert review

- Anomaly ≠ failure.
- A rare state may be startup, maintenance, or a collection error.
- Expert review and context are essential.
- Rank candidates by their anomaly scores.



End-to-End Example: Industrial Equipment

50 sensors → operating modes → 2D map → unusual states

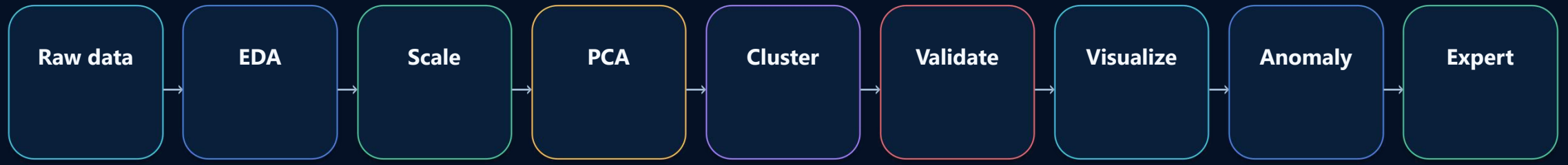


INTERPRETATION

Clusters may represent normal load, high load, startup, or shutdown. Isolation Forest then highlights rare states within or between operating modes. An engineer checks what these mean physically.

Complete Lecture 9 Workflow

How to combine methods in one data-science process



Unsupervised learning is not just one fit() command. It is an exploratory cycle: prepare → discover structure → test stability → interpret → revisit the data.

Summary and Review Questions

Key takeaways from the lecture

1 • CLUSTERING

K-Means: centroids.
DBSCAN: density.

Elbow and silhouette help, but do not replace domain knowledge.

2 • DIMENSIONALITY

PCA: compression and variance.
t-SNE/UMAP: exploration.

Interpret projections with caution.

3 • ANOMALIES

Isolation Forest: ease of isolation.

An anomaly is a candidate for investigation, not an automatic diagnosis.